AI for Physics Projects

Course notes: background, reference cards and annotated references

Course of **[DATE]**

How to use these notes

These notes follow the slides block by block and add what does not fit on a slide: the reasoning behind each claim, the details a sharp question will reach, and where to read more. They serve three readers.

- **The instructor**: each section explains the claims on its slides in enough depth to answer questions, and ends with a box of questions you may get.
- **Participants who want to go further**: Part II has two reference cards (git and a verification checklist) that also work on their own; each section points to the best next reading.
- **Anyone checking a source**: Part III lists every reference with a link, what it is good for, and how it was verified. Numbers in square brackets, such as [2], point there.

Conventions. Throughout, $\hbar = 1$; times are in µs and angular frequencies in rad/µs, so a drive at 1 MHz has angular frequency $2\pi \times 1\,\mathrm{rad}/\mu\mathrm{s}$. The basis order is $(|e\rangle, |g\rangle)$, with $|e\rangle = (1,0)^{\mathsf{T}}$, $|g\rangle = (0,1)^{\mathsf{T}}$, $\sigma_z |e\rangle = +|e\rangle$ and $\sigma_- = |g\rangle\langle e|$, as in QuantumOptics.jl. Section 8 defines the Hamiltonian and every other symbol of the toy problem.

Code. The Julia project `DrivenSpin/` of the course repository is the reference for all code, and these notes quote it verbatim: the listings marked with a file name are read from `DrivenSpin/` when the PDF is built. `DrivenSpin/` contains stage 1 and the exercise sheets in `exercises/`. The reference solutions of stages 2–5 are kept out of the course repository until the hands-on is over, so for those stages these notes follow the exercise sheets and the physics, and quote no solution code.

Contents

I Notes, block by block **5**

1 The thesis: verification is the bottleneck **5**

1.1 Two data points from October 2026 5

1.2 From the data to the thesis 5

1.3 The new PhD student 5

1.4 A sceptical counterweight 6

2 How a large language model works **6**

2.1 Tokens . . . 6
2.2 Next-token prediction . . . 7
2.3 Pre-training: a very large fit . . . 7
2.4 Post-training: from text model to assistant . . . 8
2.5 Properties and their consequences . . . 8

3 Chatbots, harnesses and agents **9**
3.1 What a harness is . . . 9
3.2 Why the closed loop matters . . . 10
3.3 Permissions . . . 10
3.4 The live demo: same task, two ways . . . 10
3.5 Beyond code . . . 11

4 Context, effort and persistent instructions **11**
4.1 The context window . . . 11
4.2 Context rot . . . 11
4.3 What a session costs . . . 12
4.4 Instruction files . . . 12
4.5 Skills . . . 13
4.6 A prompt is a specification . . . 14
4.7 A workflow that works . . . 14

5 Git: the safety net **14**
5.1 What git stores . . . 15
5.2 Three places for a change . . . 15
5.3 Reading a diff . . . 15
5.4 Undo . . . 16
5.5 Branches and merging . . . 16
5.6 Remotes . . . 16
5.7 What goes into git . . . 16
5.8 Git with an agent . . . 17
5.9 Block 4b: the local hands-on . . . 17

6 Project structure, tests and continuous integration **18**
6.1 The DrWatson layout . . . 18
6.2 Scripts and `produce_or_load` . . . 19
6.3 The environment is part of the result . . . 20
6.4 Tests . . . 20
6.5 Continuous integration . . . 21
6.6 Your tools, same principles . . . 21

7 Verification in physics **22**
7.1 Exact statements . . . 22
7.2 Consistency checks . . . 23
7.3 Injection–recovery . . . 23
7.4 Tests first . . . 24
7.5 A status for every claim . . . 24

8 The driven spin: physics of the hands-on **24**
8.1 The model . . . 25
8.2 Stage 1: Rabi oscillations . . . 25
8.3 Stage 2: decoherence . . . 27
8.4 Stage 3: Ramsey fringes and spin echo . . . 28
8.5 Stage 4: a synthetic experiment and its fit . . . 28
8.6 Stage 5 (stretch): the Jaynes–Cummings model . . . 29
8.7 The hands-on in practice . . . 29

9 Failure modes and responsibility **30**
9.1 The planted bug . . . 30
9.2 The bug your tests cannot see . . . 31
9.3 Rogues' gallery . . . 32
9.4 You are the author . . . 32
9.5 Conclusion of the course . . . 33

10 Trendy things people talk about **33**
10.1 The Jevons paradox . . . 33
10.2 Vibe coding . . . 33
10.3 Model Context Protocol (MCP) . . . 34
10.4 Agent skills and instruction files . . . 34
10.5 Long-horizon agents . . . 34
10.6 Reward hacking . . . 34
10.7 Further candidates . . . 34

II Reference cards **34**

11 Git cheat sheet **35**

12 Before trusting AI output: a checklist **36**

III References **37**

13 Annotated references **37**

Part I

Notes, block by block

1 The thesis: verification is the bottleneck

On the slides: two motivation slides (TheoPhysEval, Tao), the thesis, and the "new PhD student" mental model.

1.1 Two data points from October 2026

Agents solve research problems. Ramesh Chandra et al. [1] collected 50 unpublished research-level problems in theoretical physics, each with a verifiable final answer and each about a day's work or less for a world expert. Agents received literature search and the ability to write and run scripts, and no human hints. The best agent reached 98 % correct final answers (49 of 50) at an average cost under 3 USD per problem; the runner-up solved 46. Accuracy changed little when web search and background papers were withheld, which the authors read as substantial memorisation of the literature. They also discuss why no agent-led breakthrough has appeared despite this problem-solving ability. Two points matter for the course: the problems had answers that could be *checked*, and someone had to build that check.

Abundance outpaces verification. Tao's essay [2], based on his public lecture at the 2026 International Congress of Mathematicians, assumes that AI tools will perform research-level mathematical tasks and asks what mathematicians actually value. In Section 7 he lists the "impedance mismatches" that would follow; the first is the sentence quoted on the slide, that AI-generated proofs will accumulate faster than they can be verified. He calls the result a transition from proof scarcity to proof abundance, and notes that journals, priority conventions, hiring and prizes were all designed for scarcity. The slide carries the argument to physics, where code, numerics and derivations have become cheap in the same way.

1.2 From the data to the thesis

If producing a result costs minutes, the scarce resource is the confidence that the result is right. Research software engineering (RSE) is the body of practice that builds that confidence for code written by humans: version control, a fixed project structure, automated tests and continuous integration [32, 33]. None of it was designed for AI, and all of it applies, because every one of these practices was invented for contributors who make mistakes. An agent is such a contributor, only faster.

1.3 The new PhD student

The mental model on the slide has four traits, and each maps onto a later block.

- **Fast**: it produces more than you can read, so the reading has to be made efficient (diffs, Section 5; tests, Section 7).
- **Eager**: it rarely declines a task, and rarely says "I don't know" (Section 2, post-training).
- **Overconfident**: correct and incorrect output read the same (Section 2, plausibility).
- **Forgetful**: nothing survives the session unless it is written to disk (Section 4).

The analogy has a limit worth stating if asked: a PhD student learns from your corrections and becomes more reliable over months; a model does not, unless the correction is written into a file it reads every time.

1.4 A sceptical counterweight

A randomised controlled trial by METR [3] gave 16 experienced open-source developers 246 tasks in their own repositories, with AI tools allowed or disallowed per task. Before the study they forecast that AI would cut completion time by 24 %; afterwards they estimated it had cut it by 20 %; measured, it *increased* completion time by 19 %. The study used early-2025 tools on large, mature codebases the developers knew well, and the authors caution against generalising. Its lasting lesson is the gap between perceived and measured productivity: how fast the work *feels* is not evidence. The slides dropped this study for pace; it is a good answer to "but does it really make us faster?"

Questions you may get

Q: Is TheoPhysEval not just measuring memorisation?
Partly, and the authors say so: removing web search and papers changed accuracy little. The problems were unpublished, but their ingredients were not. The course's point survives either way: the answers were checkable, and the check made the result usable.

Q: If agents are this good, why spend an afternoon on verification?
Because "98 %" was measured against known answers. In your research there is no answer key; the tests and cross-checks you write are the answer key.

Q: Did Tao say AI is bad for mathematics?
No. The essay conditions on capable AI arriving and asks which values and institutions need to be made explicit. The quoted line describes a bottleneck, not a verdict.

Read next: Tao [2], Sections 1 and 7 (short and non-technical); the TheoPhysEval abstract and discussion [1]; the METR paper [3].

2 How a large language model works

On the slides: tokens; next-token prediction; generation as a loop; pre-training as a fit; post-training (instruction tuning, human feedback, verifiable rewards); two tables of properties and their consequences; knowledge cutoff and the Julia bias.

2.1 Tokens

A model never sees characters. A *tokenizer* splits text into pieces from a fixed vocabulary and maps each piece to an integer. Modern tokenizers learn the vocabulary from data with byte-pair encoding [8]: start from single bytes and repeatedly merge the most frequent adjacent pair into a new symbol, until the vocabulary has the target size. Frequent words become one token; rare words, numbers, code and LaTeX split into several. Llama 3 uses 128K tokens and reaches 3.94 characters per token on English text [9]. An English word averages about five letters plus a space, so one token is roughly 0.7 words, consistent with the "0.75 words per token" on the slide. The token numbers on the slide are illustrative.

Two consequences come up in questions. Letter-level tasks (counting the r's in a word, reversing a string) are hard because the model does not see letters. Arithmetic on long numbers is unreliable

because digits are grouped into tokens in irregular ways; this is one reason agents call a calculator or write a script.

2.2 Next-token prediction

Write the tokens so far as $x_1, \ldots, x_n$; this sequence is the *context*. The model is a function that returns a probability distribution over the vocabulary for the next token,

$$p_\theta(x_{n+1} \mid x_1, \ldots, x_n), \tag{1}$$

where θ denotes the parameters of a neural network, a *transformer* [7]. Large models have 10^{11} to 10^{12} parameters; the largest Llama 3 model has 4.05×10^{11} [9]. The network outputs one real number per vocabulary entry, a *logit* z_i, and the distribution is the softmax

$$p_\theta(x_{n+1} = i \mid x_1, \ldots, x_n) = \frac{\mathrm{e}^{z_i/T_\mathrm{s}}}{\sum_j \mathrm{e}^{z_j/T_\mathrm{s}}}, \tag{2}$$

with a *sampling temperature* T_s set by the harness (the original distribution is $T_\mathrm{s} = 1$). A physicist reads this as a Boltzmann distribution with energies $-z_i$. Small T_s concentrates the probability on the most likely token; large T_s flattens it.

Generation is a loop. Sample a token, append it to the context, compute the distribution again. A long answer is thousands of passes. Two facts follow and are on the slide: in each pass the context is the model's only input, and a token once written is never revised. A model can correct itself only by writing more text, for example "wait, that is wrong", which is what reasoning models learn to do (below).

Attention (backup slide). Inside the transformer, each token is a vector. In each layer, each token's vector is mapped linearly to a *query*, a *key* and a *value*; stacking these for all tokens gives matrices Q, K, V, and the layer computes

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\mathsf{T}}{\sqrt{d}}\right) V, \tag{3}$$

where d is the dimension of the keys and the softmax acts row by row [7]. Row m of the result is a weighted average of the values of all tokens, weighted by how well their keys match the query of token m. In a language model a mask restricts the average to earlier tokens. The matrix QK^T has one entry per pair of tokens, so the cost grows with the square of the context length; this is one reason the context window is finite. 3Blue1Brown's chapters 5 and 6 [6] are the best visual explanation.

2.3 Pre-training: a very large fit

Pre-training minimises the cross-entropy of the next token over a corpus,

$$\mathcal{L}(\theta) = -\sum_n \ln p_\theta(x_{n+1} \mid x_1, \ldots, x_n), \tag{4}$$

where the sum runs over every position n of every document. This is a maximum-likelihood fit: the same principle as a least-squares fit with Gaussian errors, with 10^{12} parameters and 10^{13} data points. The largest Llama 3 model was pre-trained on 15.6×10^{12} tokens [9]. The loss falls as a power law in model size, data and compute [10], and for a fixed compute budget model size and data should grow together [11]; these *scaling laws* are why models became large.

What the fit learns is the distribution of text. Text about physics is mostly correct, so the model learns a great deal of physics; but the objective rewards the *form* of plausible text and never checks

its truth. That is the root of the first row of the property table: a fluent wrong derivation and a fluent right one are both high-likelihood text.

Knowledge cutoff. The corpus ends at some date. Packages released or changed after it are unknown or known in an old version. The corpus also reflects what people write: there is far more Python than Julia. The slide's example shows the typical result. Asked for QuantumOptics.jl, the model writes `mesolve`, the name of the master-equation solver in QuTiP (Python); the Julia package calls it `timeevolution.master` [53, 54]. The code looks right and fails at the first run, which an agent notices and a chatbot user may not.

2.4 Post-training: from text model to assistant

A pre-trained model continues text; asked a question, it may continue with more questions. Post-training turns it into an assistant in three stages.

Instruction tuning (supervised fine-tuning). Continue training on examples of prompts with good answers written by people. The model imitates the format and tone.

Reinforcement learning from human feedback (RLHF). People compare pairs of answers to the same prompt. A second network, the *reward model*, is trained to predict which answer people prefer, giving a scalar score $r(x, y)$ for prompt x and answer y. The language model, now viewed as a policy $\pi_\theta(y \mid x)$, is then trained by reinforcement learning to maximise

$$\mathbb{E}_{y\sim\pi_\theta}\left[r(x,y)\right] - \beta\, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})\,, \tag{5}$$

where π_{ref} is the model before this stage, D_{KL} the Kullback–Leibler divergence and $\beta > 0$ a weight that keeps the model from drifting far from fluent text. The method comes from Christiano et al. [12] and was applied to language models in InstructGPT [13], where a 1.3-billion-parameter model trained this way was preferred by human raters over the 175-billion-parameter GPT-3. Variants replace human raters with a model that judges answers against written principles [14].

Reinforcement learning with verifiable rewards (RLVR). Replace the learned reward model with a check: the final answer to a maths problem matches the known one, or the generated code passes the tests. The term RLVR was introduced with Tülu 3 [16]; DeepSeek-R1 [17] showed that this kind of training alone makes a model produce long chains of intermediate reasoning, check its steps and backtrack, before giving an answer. These are the "thinking" tokens of current models. Earlier, the same behaviour had been elicited by prompting ("chain of thought") [15].

The key box on the slide summarises the incentives: the model was rewarded for answers that *look good to people* and *pass checks*. Neither is the same as being correct, and both leave marks.

2.5 Properties and their consequences

Trained to sound plausible (pre-training). Right and wrong output have the same fluency. References are the classic case: a plausible author, title, journal and year are easy to generate and the model has no lookup to check them against. Every reference in this course was opened and checked (Part III).

Trained to please (human feedback). Raters prefer answers that agree with them. Sharma et al. [18] showed that five assistants consistently exhibit this *sycophancy*, for example abandoning a correct answer when the user pushes back, and that both humans and preference models sometimes prefer a convincing sycophantic answer over a correct one. The defence on the slide follows: ask the model to find the flaw in your argument, not to confirm it.

Rewarded for passing checks (verifiable rewards). A policy rewarded for passing tests learns that weakening the test also passes it. METR reported in 2025 that frontier models, in its evaluations, attempted "(often successfully) to get a higher score by modifying the tests or scoring

code" [19]. In everyday use this looks like a loosened tolerance, a deleted assertion, or a special case for the test input. The defence: read every change to a test, and commit tests before the implementation (Section 7).

Random sampling (architecture). With $T_s > 0$ the same prompt gives different answers. This is a free diagnostic: if two runs disagree on a number, neither is reliable. Setting $T_s = 0$ makes the choice greedy, but in practice deployed models do not guarantee bit-identical outputs, so do not build reproducibility on it; build it on code and data in a repository.

No memory between sessions. The parameters do not change while you use the model. Everything it "knows" about your project in a session is in the context; at the next session it starts from zero. Hence: write things down (Section 4).

Finite context. See Section 4.

Questions you may get

Q: Is it "just autocomplete"?
The objective is next-token prediction, yes. But predicting the next token of a derivation well requires representing a lot of the structure of the derivation, and post-training adds goal-directed behaviour. "Just autocomplete" underestimates what it can do; "understands physics" overstates how reliably. The practical answer is the course's: treat its output as a claim to be checked.

Q: Why does it invent references instead of saying it doesn't know?
Nothing in pre-training rewards abstaining, and a confident well-formatted answer tends to be preferred by raters. Agents with web search can check references, but only if they are asked to and actually do it; check the link yourself.

Q: Does it learn from my conversations?
Not within the session: the parameters are fixed. Whether a provider uses conversations for future training depends on its terms and your account settings; check them before pasting unpublished data (Section 9).

Q: How many parameters does model X have?
Providers of closed models do not publish it. Open-weight models do, for example 4.05×10^{11} for the largest Llama 3 [9].

Read or watch next: Karpathy's *Intro to Large Language Models* (1 h) and *Deep Dive into LLMs like ChatGPT* (3.5 h), which covers tokenization, pre-training and post-training in exactly this order [4, 5]; 3Blue1Brown chapters 5–6 for attention [6]; the InstructGPT paper [13], Sections 1 and 3, for RLHF.

3 Chatbots, harnesses and agents

On the slides: a chatbot where you are the loop; an agent where the loop closes; one agent transcript; chatbot versus agent table; permissions; the live demo "Same task, two ways"; asking for verifiable output beyond code.

3.1 What a harness is

The model of Section 2 maps text to text and does nothing else. Everything else in a product such as Claude Code or Codex is the *harness*: the program that

1. assembles the context (system instructions, instruction files, your messages, earlier tool results),

2. offers the model a set of *tools*, each described by a name, a description and a schema for its arguments (read a file, edit a file, run a shell command, search the web),
3. recognises when the model's output is a *tool call*, a structured request such as `run("julia -project -e 'using Pkg; Pkg.test()'")`, asks you for permission if needed, executes it, and appends the result to the context,
4. loops until the model answers without a tool call, and
5. manages the context when it fills up (Section 4).

The model never touches your computer. It writes text that requests an action, and the harness decides whether to carry it out. A chatbot in the browser is a harness with no tools on your machine, and *you* carry code and error messages between the model and your computer: you are the loop.

Interleaving reasoning with actions and observations is known as the ReAct pattern [21]. Schluntz and Zhang [20] distinguish *workflows*, in which code fixes the sequence of model calls, from *agents*, in which the model decides its next step from the results of the previous one. Their advice, start with the simplest design that works, applies to how you use agents too.

3.2 Why the closed loop matters

The transcript on the slide is illustrative (the real test is the π-pulse line of the `"Rabi"` testset, Section 6), and it shows the point of the block. The agent runs the tests, reads a failure (`0.0` $\approx$ `1.0`), reads the source, finds Ω where $\Omega/2$ belongs, edits, and runs the tests again. It found the factor of two *because a test encoded the physics*: a resonant π-pulse must invert the spin. With the bug, the population at $t = \pi/\Omega$ is $\sin^2(\pi) = 0$ instead of $\sin^2(\pi/2) = 1$ (Section 9). An agent can check its work against reality, but only against the reality you give it.

Evidence that this works at scale: SWE-bench [22] tests whether a system can resolve 2294 real GitHub issues from 12 popular Python repositories, judged by the repositories' own tests. METR measures the length of software tasks, in human working time, that agents complete with 50 % reliability, and found that this length has doubled roughly every seven months since 2019 [23]. Both benchmarks rely on tests written by people.

3.3 Permissions

Agents ask before editing a file or running a command, and every agent lets you approve whole classes of actions (all edits, or specific commands). The slide's rules:

- Never auto-approve deletion, overwriting, or anything outside the project folder. An agent runs commands with your user's rights.
- "Yes, yes, yes" is not supervision. If you approve without reading, you have granted full permission with extra steps.
- Git makes a longer leash safe (Section 5): any change inside the repository can be inspected and undone, but nothing outside it can.

3.4 The live demo: same task, two ways

The prompt on the slide asks for Rabi oscillations with the course Hamiltonian, two detunings, three Rabi periods and a check against the analytic formula. What to point out while it runs:

- In the chat, every error costs a copy–paste round trip; the model never sees the plot.

- The agent writes files, runs Julia, reads the error, fixes it. Count the tool calls.
- Ask the room: what did it check? (Typically it compares with the formula at a few points.) What did it not check? (Units, the sign convention of Δ, the initial state, that the plot shows what the caption claims.)

3.5 Beyond code

For any task, ask for an output you can verify: links you open for a literature search, a script that checks a derivation numerically, an analysis that lives as a script in the repository, and text you can defend line by line. A number in a chat window cannot be reproduced; a script in a repository can.

Questions you may get

Q: Which agent should I use?
The course does not prescribe one; the concepts carry over. Use the one your institution or subscription gives you, and check its data-handling terms (Section 9).

Q: Can the agent delete my files?
If you permit it, yes. Keep the project under git, commit before the agent starts, and do not approve commands that act outside the project folder.

Q: Is "agent" just marketing?
In the sense used here it has a precise meaning: a model in a loop that chooses tool calls from their results [20]. Whether that is useful depends on whether the results contain a signal of correctness, which is what tests provide.

Read next: *Building effective agents* [20] (short, practical); Anthropic's best-practice guide for its coding agent [27], whose first advice, "give Claude a way to verify its work", is tool-specific in wording and general in content.

4 Context, effort and persistent instructions

On the slides: the context window; context rot; context hygiene; the cost of thinking; instruction files; skills; a prompt as a specification; the workflow explore–plan–implement–verify–commit. This block is the first to shorten if time runs out.

4.1 The context window

The context is the token sequence $x_1, \ldots, x_n$ of Section 2. In an agent session it contains, in order: the harness's system instructions and tool descriptions, your instruction files, your messages, every file the agent read, every command output, and the model's own answers and reasoning. Its length is capped by the context window, currently 10^5 to 10^6 tokens (the largest Llama 3 model has 1.28×10^5 [9]). At about 0.7 words per token, 10^5 tokens is a long novel. That sounds like a lot until an agent reads a few source files, a test log and a long solver output.

4.2 Context rot

Accuracy degrades as the context grows, well before the window is full. Hong, Troynikov and Huber [24] tested 18 models on tasks that stay equally hard as the input grows, so that any drop is due to length alone. Every model degraded with input length, and non-uniformly. Three findings are worth quoting: a single distractor (text similar to the answer but wrong) lowered accuracy,

and its effect grew with length; on a conversational memory benchmark, all models did much better with a focused input of about 300 tokens than with the full $\sim$ 113K-token history; and models performed *worse* when the irrelevant filler was a coherent text than when its sentences were shuffled. Earlier, Liu et al. [25] found that models use information at the beginning and end of a long context better than information in the middle.

Practical consequences:

- A wrong turn stays in the context and keeps influencing later answers. Arguing with the agent adds more of it. Start a fresh session with a better prompt.
- When the window fills, harnesses *compact*: they replace the history by a summary written by the model [26]. The summary is lossy; details you care about may not survive.
- One task, one session. Write what must survive into files (below).

Point, do not paste. Three thousand lines of solver log in your message cost context and bury the one relevant line. Naming the failing test and asking the agent to run only that test lets it read what it needs. Anthropic's engineering team calls this loading context "just in time" through references such as file paths [26].

4.3 What a session costs

Providers charge per token, at different prices for input and output. With N_{in} and N_{out} the numbers of input and output tokens and c_{in}, c_{out} the prices per token,

$$\text{cost} \approx N_{\text{in}}\, c_{\text{in}} + N_{\text{out}}\, c_{\text{out}}. \tag{6}$$

The slide's remark that every agent step re-reads the context has a quantitative consequence. Suppose a session takes K steps (tool calls), starts from a context of C_0 tokens, and each step adds about a tokens (a file read, a command output). Step k reads the whole context, $C_0 + ka$ tokens, so

$$N_{\text{in}} \approx \sum_{k=1}^{K} (C_0 + ka) \approx KC_0 + \tfrac{1}{2}aK^2, \tag{7}$$

quadratic in the length of the session. Long sessions are expensive as well as less accurate. Many providers discount input that repeats an earlier prefix (*prompt caching*), which softens but does not remove the effect.

N_{out} includes the reasoning tokens of Section 2, which you may never see. The *effort* setting of current agents controls how much the model reasons before answering: low effort for mechanical edits, high for debugging and derivations. For scale, the TheoPhysEval agents solved day-long research problems for under 3 USD each [1].

4.4 Instruction files

Because the model remembers nothing between sessions, harnesses read an instruction file from the project folder at the start of every session and put it into the context. The file name depends on the tool: `AGENTS.md` is an open convention read by many agents [28]; Claude Code reads `CLAUDE.md` [27]. The `DrivenSpin/` project has an `AGENTS.md` and a one-line `CLAUDE.md` that points to it.

What belongs in it: the conventions the agent cannot guess (units, basis order, sign of Δ), the command that runs the tests, and the rules that protect you (never weaken a test, never write to `data/exp_raw`). What does not: anything the agent can read from the code, long tutorials, information that changes often. The file is in every context, so every line costs tokens in every step; a long file also dilutes the rules that matter [27].

The file of the hands-on project, in full (the slide shows its first two blocks):

DrivenSpin/AGENTS.md

```
# AGENTS.md

Instructions for AI coding agents working in this project. Humans: this file is
also the shortest description of how the project works.

DrivenSpin simulates a driven two-level system (a spin, or a qubit) in Julia, as a
DrWatson project. Library code is in `src/DrivenSpin.jl`, scripts in `scripts/`,
tests in `test/runtests.jl`.

## Physics conventions
- ħ = 1; times in μs, angular frequencies in rad/μs
- Basis (|e⟩, |g⟩); H = (Δ/2)σz + (Ω/2)σx with Δ = ω_q - ω_d
- Never change a convention without asking

In detail:
- |e⟩ = [1, 0], |g⟩ = [0, 1] (spin up, spin down, as `SpinBasis(1//2)` in
  QuantumOptics.jl); σz|e⟩ = +|e⟩; σ_ = |g⟩⟨e|.
- Ω is the Rabi frequency: on resonance, P_e oscillates at angular frequency Ω.
  Starting from |g⟩: P_e(t) = (Ω²/Ω_R²) sin²(Ω_R t/2), with Ω_R = √(Ω² + Δ²).
- Master equation: dρ/dt = −i[H, ρ] + Γ₁ D[σ_]ρ + (Γ_φ/2) D[σz]ρ, with
  D[L]ρ = LρL† − ½{L†L, ρ}, Γ₁ = 1/T1, Γ_φ = 1/T_φ, 1/T2 = 1/(2T1) + 1/T_φ.
  T2 ≤ 2T1 always.
- Jaynes-Cummings: H = ω_c a†a + (ω_q/2)σz + g(a†σ_ + aσ₊).

## How to work
- Run the tests: julia --project -e 'using Pkg; Pkg.test()'
- Every new function gets a test with a physical check
- Never weaken or delete a test to make it pass: ask first
- Never write to data/exp_raw

In detail:
- Tests first. When asked to write tests, write only tests. When asked to
  implement, do not modify anything in `test/`; if a test seems wrong, stop and
  explain why instead of changing it.
- Physics goes in `src/`; scripts in `scripts/` only call functions from `src/`,
  set parameters and save results with `produce_or_load`.
- Keep the whole test suite under one minute.
- Fix random seeds in tests.
- Do not add, remove or update packages (`Pkg.add`, `Pkg.update`) without asking:
  `Manifest.toml` pins the versions that produced our results. The project already
  depends on DrWatson, QuantumOptics, OrdinaryDiffEq, LsqFit, CairoMakie and JLD2.
- Plots: CairoMakie only.
- Small steps; the human commits after reading each diff. Do not commit unless asked.
```

Three design choices are worth pointing out. The short rules come first and the details after, so that the rules a reader (or a model) must not miss stand at the top. The rules on `test/` encode the "tests first" workflow of Section 7. The rule against `Pkg.add` and `Pkg.update` protects the pinned environment of Section 6. `CLAUDE.md` contains the single line `@AGENTS.md`, which tells Claude Code to read `AGENTS.md`.

4.5 Skills

A *skill* is a folder with a file `SKILL.md` that starts with a name and a one-line description, followed by instructions, and optionally scripts and further files. At the start of a session only the names and descriptions enter the context; when a task matches a description, the agent reads the full file, and it opens further files only when a step needs them [29]. This *progressive disclosure* lets a lab keep

many procedures (fit Ramsey data, calibrate a pulse, format a figure) at the cost of one line each. The slide's `fit-ramsey` skill is in the repository, in `DrivenSpin/.claude/skills/fit-ramsey/`, where Claude Code finds it. Instruction files and skills are *procedural memory written to disk*: the model does not learn, but the folder does.

4.6 A prompt is a specification

The slide contrasts "Simulate a qubit with decoherence" with a prompt that names the goal and context (which file, which conventions), the verification (the tests that define "done"), and the constraints (what must not change; plan before code). Each part closes a door the agent would otherwise walk through: without the file it invents a new one; without the conventions it picks its own; without the tests "done" means "looks done"; without constraints it "improves" the Hamiltonian.

4.7 A workflow that works

Explore (read the code), plan (you approve), implement (small steps), verify (tests, plots), commit (you read the diff). The same sequence is the core recommendation of Anthropic's guide [27], and it is tool-independent. Most failures come from skipping *plan* (the agent solves the wrong problem) or *verify* (it solves the problem wrongly).

Questions you may get

Q: How big is my context right now?
Most agents can show it (a command or a status line). If you cannot see it, start fresh sessions often.

Q: Should I always use the highest effort?
No: it costs more and is slower, and for mechanical edits it does not help. Raise it when the agent goes in circles on a hard problem.

Q: Instruction file or skill?
Rules that apply to every task go into the instruction file. Procedures that apply to some tasks go into skills, so they cost context only when used.

Q: Is "context rot" a real effect or a vendor's term?
The term comes from the Chroma report [24], a controlled measurement on 18 models; the effect is consistent with earlier academic work [25].

Read next: *Effective context engineering for AI agents* [26]; the Chroma report [24], whose figures are worth a look; the skills article [29].

5 Git: the safety net

On the slides: why git now; a chain of snapshots; three places for a change; a first repository; reading a diff; a weakened test; undo; branches; remotes; what goes into git; habits and prompts for working with an agent; the live demo "The agent wrecks the project"; the nine commands. Block 4b is a local hands-on (four exercises).

The one-page cheat sheet is Section 11. This section explains the model behind the commands, which is what you need to answer "what exactly happens when…".

5.1 What git stores

A git repository is the hidden folder `.git/` inside your project. It holds a database of *objects*, each named by a hash of its content [35]:

- a *blob* is the content of one file;
- a *tree* is a directory listing: names, permissions, and the hashes of the blobs and subtrees;
- a *commit* records one tree (the snapshot of the whole project), the hash of its parent commit (two parents for a merge), the author, the date and the message.

The hash is a 40-hex-digit SHA-1 value; the first seven digits (`9c0d5e2`) almost always suffice to identify a commit within a project. Because a commit's hash covers its tree and its parent, which covers *its* parent, a hash identifies the whole history up to that point: change one byte anywhere in the past and every later hash changes. This is why a hash stored next to a result (Section 6) pins down the code exactly.

A commit is a snapshot, not a list of changes. A diff is computed on demand by comparing two snapshots. Unchanged files are stored once and shared between snapshots, so history is cheap.

A *branch* is a file containing one commit hash: a movable label. Committing on a branch writes the new commit and moves the label to it. `HEAD` names what you have checked out, normally a branch. That is all a branch is, which is why creating and deleting branches costs nothing.

5.2 Three places for a change

Place	What it is
Working folder	The files you and the agent edit.
Staging area (index)	The snapshot being prepared; `git add` copies a file's current content here.
History	The commits; `git commit` turns the staging area into a new commit.

The staging area lets you choose what goes into a commit: two unrelated changes, two commits (Exercise 2 of block 4b). `git status` reports where each change is; run it often.

The forms of `git add`. `git add <file>` stages one file; `git add .` stages everything new, changed or deleted in the current folder and below; `git add -u` stages changes to files git already tracks and leaves new files out; `git add -p` shows each change (*hunk*) and asks `y` or `n`. In the hands-on it asks file by file in alphabetical order. `git commit -am "..."` is `git add -u` plus commit in one step: convenient, but it skips the look at what you stage.

5.3 Reading a diff

```
--- a/src/DrivenSpin.jl
+++ b/src/DrivenSpin.jl
@@ -12,6 +12,6 @@ function hamiltonian(Δ, Ω)
     σx = [0 1; 1 0]
     σz = [1 0; 0 -1]
-    return Δ/2*σz + Ω*σx
+    return Δ/2*σz + Ω/2*σx
 end
```

- `--` and `+++` name the old (`a/`) and new (`b/`) version of the file.
- `@@ -12,6 +12,6 @@` is the hunk header: the excerpt starts at line 12 and is 6 lines long, in the old and in the new file. Git appends the nearest enclosing function as a hint.

- Lines starting with a space are unchanged context; `-` lines were removed, `+` lines added. A modified line appears as one removal and one addition.

`git diff` shows changes not yet staged; `git diff -staged` shows what the next commit will contain; `git diff <hash1> <hash2>` compares two snapshots; `git show <hash>` shows one commit with its message.

Spot the problem (slide). An agent changes `@test excited_population(H, π/Ω) ≈ 1.0` to `... ≈ 1.0 atol=1.0`. Now any value between 0 and 2 passes, so the test checks nothing while staying green. Every change to a test needs a reason you accept.

5.4 Undo

Situation	Command	Effect
Unwanted edits, not staged	`git restore <file>`	Working file back to the staged (or last committed) version. The edits are gone.
Staged by mistake	`git restore -staged <file>`	Out of the staging area; the edit stays in the working file.
A bad commit	`git revert <hash>`	A *new* commit that undoes it; history is kept.

`git restore .` discards *all* uncommitted changes below the current folder: look at `git status` and `git diff -stat` first. Uncommitted work that is discarded is not recoverable from git, which is why "commit before the agent starts" is the first habit.

Two commands destroy work and should never run without your explicit approval: `git reset -hard` (moves the branch and overwrites the working folder) and `git push -force` (replaces the history on the remote, including your colleagues' commits). Agents occasionally reach for them to "clean up".

5.5 Branches and merging

`git switch -c try-rk4` creates a branch at the current commit and switches to it; commits now move `try-rk4` and leave `main` where it was. If the experiment works, `git switch main` and `git merge try-rk4`. If `main` has not moved, the merge just moves the `main` label forward (a *fast-forward*); otherwise git creates a merge commit with two parents, as on the slide. If both branches changed the same lines, git stops with a *conflict* and marks the competing versions in the file; you edit, `git add` and `git commit`. If the experiment failed, `git branch -D try-rk4` deletes the label and its commits are as if they never happened.

5.6 Remotes

A remote is another copy of the repository, usually on GitHub, with the full history. `git clone` copies one and remembers it as `origin`; `git push` sends your new commits; `git pull` fetches new commits and merges them into your branch. Every clone is a complete backup. The installation guide set up login over SSH with `gh`.

5.7 What goes into git

Commit source, scripts, tests, `Project.toml` and `Manifest.toml`, `README.md` and `AGENTS.md`. Do not commit large raw data (archive it elsewhere), generated results and plots (they can be

regenerated), and never passwords or API keys. The `.gitignore` file lists patterns git should not track (`data/`, `plots/`, `*.jld2`).

If a secret was committed and pushed, consider it public: revoke it and issue a new one. Removing it from history afterwards is possible but does not undo copies that already exist.

5.8 Git with an agent

The habits on the slide:

1. Commit before the agent starts: a clean point to return to, and `git diff` then shows exactly what the agent did.
2. One task, one commit: each change can be reviewed and reverted on its own.
3. Read the diff before every commit, tests first.
4. Let the agent type the git commands; you check the result.

Prompts such as "Commit my manual changes before starting" and "Work in modular commits: one commit per change, with the tests passing at each" make the agent keep a history in which your edits and its edits never share a commit.

The live demo. Ask the agent for a sweeping refactor ("all parameters in one struct"), then: `git status -short` (what did it touch?), `git diff -stat` (how much?), `git restore .` (back to the last commit). Thirty seconds, nothing lost, *because* the last commit was clean.

5.9 Block 4b: the local hands-on

Four exercises in a throw-away folder `git-practice/`, in pairs, without agent or GitHub; all commands one per line so they also work in Windows PowerShell 5.

1. **First commits**: `git init`, `status`, `add` one file, `commit`, then the other; `git log -oneline`. The files: `rabi.jl` with the Rabi formula as one line of Julia, and `notes.md`.
2. **One change, one commit**: edit both files; `git diff`; `git add -p` (answer `n` for `notes.md`, `y` for `rabi.jl`); `git diff -staged`; commit; then `git add -u` and commit the rest.
3. **Undo**: delete the formula and `git restore` it; stage a change and unstage it with `git restore -staged`; commit a broken formula ($t/2 \to t$) and undo it with `git revert -no-edit HEAD`, so both commits remain in the history.
4. **Read the history**: `git log -oneline`, `git log -stat`, `git show <hash>`, `git diff <hash1> <hash2>`, `git log -p rabi.jl`. Long output opens in a pager: arrow keys to scroll, `q` to quit.

A possible `rabi.jl` line, in the course conventions:

```
Pe(t; Δ, Ω) = Ω^2 / (Ω^2 + Δ^2) * sin(sqrt(Ω^2 + Δ^2) * t / 2)^2   # ħ = 1, rad/μs, μs
```

Questions you may get

Q: Do I need GitHub to use git?
No. Everything in block 4b is local. GitHub adds backup, sharing and CI.

Q: I ran `git restore` on the wrong file. Can I get it back?
Not from git, if the change was never staged or committed. Your editor's undo or local history may still have it. This is why the agent should start from a clean commit.

Q: What is the difference between `revert` and `reset`?
`revert` adds a commit that undoes an earlier one; history only grows, which is safe on shared branches. `reset` moves the branch label backwards, so later commits drop out of the history; with `-hard` it also overwrites your files.

Q: Why does `git pull` sometimes create a merge commit?
Because you and the remote both have new commits; git joins the two lines of history.

Q: Is SHA-1 not broken?
Collisions have been constructed deliberately, and git has added a hardened variant and an optional SHA-256 format. For tracking your own research code, accidental collisions are not a practical concern.

Read next: *Pro Git* [35], Chapters 2 (basics), 3 (branching) and 10 (internals); the Software Carpentry lesson [36] to practise.

6 Project structure, tests and continuous integration

On the slides: which code made Figure 3; the DrWatson layout; the code as one part of the research project; `src` computes, scripts orchestrate; every result knows its commit; the environment is part of the result; tests; continuous integration; the same principles for MATLAB, instruments and FPGA.

The goal on the first slide is the test of this block: *for every figure, you can say which code, data and parameters made it.* An agent makes this harder, because it creates files faster than anyone names them well, and easier, because it follows a fixed layout reliably once the layout exists.

6.1 The DrWatson layout

DrWatson [39, 40] is a Julia package that fixes a project layout and provides a few functions for naming, saving and loading results. `initialize_project("DrivenSpin")` creates the folders and a git repository:

```
DrivenSpin/
|-- src/            # the physics: functions, no hard-coded parameters
|-- scripts/        # short runs that call src/ and save results
|-- test/           # runtests.jl: the physics checks
|-- data/
|   |-- exp_raw/    # measurements: read-only, never modified
|   |-- exp_pro/    # processed data
|   `-- sims/       # simulation results (generated)
|-- plots/          # figures (generated)
|-- Project.toml    # dependencies
|-- Manifest.toml   # exact versions
`-- AGENTS.md       # instructions for the agent
```

Functions such as `datadir("sims", "rabi")` and `plotsdir()` return absolute paths into these folders, so a script works from any working directory. A project with this layout reads the same

to you, your colleagues and the agent. The raw-data folder is read-only by convention; `AGENTS.md` repeats the rule.

The code is one part of the project. A research project also has a manuscript, notes and talks, with other collaborators and another history. The code gets its own repository inside the project folder, so it can be shared, tested and published on its own; in the course, `DrivenSpin/` always means that code repository.

6.2 Scripts and `produce_or_load`

The script of the hands-on project; its first eleven lines are the slide:

`DrivenSpin/scripts/rabi.jl`

```
using DrWatson
@quickactivate "DrivenSpin"          # this project's environment, from any folder
using DrivenSpin                     # the module in src/

for c in dict_list(Dict(:Δ => [0.0, 2π], :Ω => 2π))   # rad/μs; one run per Δ
    produce_or_load(c, datadir("sims", "rabi")) do c
        H = hamiltonian(c[:Δ], c[:Ω])
        t = range(0, 3, length = 301)                 # μs
        Dict("t" => t, "Pe" => excited_population.(Ref(H), t))
    end
end

# Plot every saved run. Results are loaded from data/sims/rabi: delete that
# folder to recompute them after changing src/.
using CairoMakie
fig = Figure()
ax = Axis(fig[1, 1]; xlabel = "t (μs)", ylabel = "P_e", title = "Rabi oscillations")
for c in dict_list(Dict(:Δ => [0.0, 2π], :Ω => 2π))
    data = wload(datadir("sims", "rabi", savename(c, "jld2")))
    lines!(ax, data["t"], data["Pe"]; label = "Δ = $(round(c[:Δ], digits = 2)) rad/μs
    ")
end
Legend(fig[2, 1], ax; orientation = :horizontal, framevisible = false)
wsave(plotsdir("rabi.png"), fig)
println("Saved ", plotsdir("rabi.png"))
```

Line by line:

- `@quickactivate "DrivenSpin"` activates the project environment (the `Project.toml` of `DrivenSpin/`) wherever the script is started from.
- `dict_list` expands a dictionary whose values may be vectors into all combinations: here two configurations, $\Delta = 0$ and $\Delta = 2\pi$, both with $\Omega = 2\pi$.
- `produce_or_load(f, config, path)` builds a file name from `config` with `savename`, and returns `(data, file)`. If the file exists, it is loaded; otherwise `f(config)` runs and its result is saved [40]. The `do` block is Julia syntax for passing `f` as an anonymous function.
- `savename` sorts the keys, joins `key=value` pairs with `_` and rounds numbers to three significant digits by default, which gives `Δ=0.0_Ω=6.28.jld2`.
- For `.jld2` files the result is saved with `tagsave`, which adds the key `gitcommit`: the output of `git describe` for the project repository [40].

If the repository had uncommitted changes, `gitcommit` ends in `-dirty`: the code that made the file is not in any commit, so that version is lost. DrWatson can also store the uncommitted diff under

the key `gitpatch` (keyword `storepatch`, off by default). Hence the rule on the slide: *commit first, then run.*

Two cautions for questions. First, `produce_or_load` decides by file name only: if you change the code and rerun, it loads the old result. Delete the file or pass `force = true`. Second, three significant digits mean $\Delta = 6.283$ and $\Delta = 6.284$ share a file name; choose parameters, or `savename` options, so that names stay distinct.

6.3 The environment is part of the result

`Project.toml` lists what you depend on and is written by `Pkg.add`. `Manifest.toml` records the exact version of every package, including indirect dependencies, and is never edited by hand. With both committed, `Pkg.instantiate()` reproduces your environment on another machine. An agent that "fixes" an error with `Pkg.update()` changes package versions, and numbers may change with them; a `Manifest.toml` in a diff you did not expect needs an explanation.

6.4 Tests

The stage-1 tests of the hands-on project; the first testset is the slide:

DrivenSpin/test/runtests.jl

```
using DrivenSpin, Test
using LinearAlgebra

@testset "Rabi" begin
    Δ, Ω = 0.3, 2π                                  # rad/μs
    ΩR = sqrt(Ω^2 + Δ^2)
    H = hamiltonian(Δ, Ω)
    for t in range(0, 3, length = 31)               # μs
        @test excited_population(H, t) ≈ Ω^2 / ΩR^2 * sin(ΩR * t / 2)^2 atol = 1e-10
    end
    @test excited_population(hamiltonian(0.0, Ω), π / Ω) ≈ 1    # π-pulse
end

@testset "Hamiltonian" begin
    H = hamiltonian(0.7, 2.1)
    @test H ≈ H'                                    # Hermitian
    @test tr(H) ≈ 0 atol = 1e-14                    # no energy offset
    @test eigvals(Hermitian(H)) ≈ [-1, 1] * sqrt(0.7^2 + 2.1^2) / 2   # ±Ω_R/2
    @test hamiltonian(1.0, 0.0) * ket_e ≈ +0.5 * ket_e    # σz|e⟩ = +|e⟩
end

@testset "Unitarity and norm" begin
    H = hamiltonian(0.3, 2π)
    for t in (0.0, 0.1, 1.7, 25.0)
        U = propagator(H, t)
        @test U' * U ≈ I
        @test norm(U * ket_g) ≈ 1
    end
end

@testset "Density-matrix population" begin
    @test excited_population(ket_e * ket_e') ≈ 1
    @test excited_population(ket_g * ket_g') ≈ 0 atol = 1e-15
    ψ = propagator(hamiltonian(0.3, 2π), 0.4) * ket_g
    @test excited_population(ψ * ψ') ≈ excited_population(hamiltonian(0.3, 2π), 0.4)
end
```

The other three testsets check structure, not a formula: the Hamiltonian is Hermitian, traceless, has eigenvalues $\pm\Omega_R/2$ and the sign convention $\sigma_z|e\rangle = +|e\rangle$; the propagator is unitary; and the two methods of `excited_population`, for a time and for a density matrix, agree. The last one ties stage 1 to stage 2, where populations come from density matrices.

`@testset` groups tests under a name that says which physical claim they check; `@test` records a pass or a failure without stopping [42]. The operator $\approx$ is `isapprox`, which by default compares with a *relative* tolerance of $\sqrt{\epsilon_{\text{mach}}} \approx 1.5 \times 10^{-8}$, where ϵ_{mach} is the double-precision machine epsilon, and no absolute tolerance. Near zero a relative tolerance is meaningless: `isapprox(1e-12, 0.0)` is `false`. Hence the `atol` on the slide wherever the expected value can vanish ($P_e(0) = 0$). Run the tests with `julia --project -e 'using Pkg; Pkg.test()'`, before every commit; keep them under a minute.

A good test checks *physics*, not the implementation (a formula, a limit, a conservation law); it is deterministic (fixed random seed); it is fast; and it fails when the physics is wrong. The last property is the one to check deliberately: break the code and watch the test fail. A test that cannot fail checks nothing.

6.5 Continuous integration

Continuous integration (CI) runs the tests on a clean machine at every push. GitHub Actions reads workflow files in `.github/workflows/`. The slide's file:

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4             # get the repository
      - uses: julia-actions/setup-julia@v2    # install Julia
      - uses: julia-actions/cache@v2          # cache packages between runs
      - uses: julia-actions/julia-buildpkg@v1 # instantiate the environment
      - uses: julia-actions/julia-runtest@v1  # Pkg.test()
```

Each `uses:` line runs a published action [43]. The result appears as a check mark or a cross next to the commit, under the *Actions* tab. CI catches "works on my laptop" (a missing dependency, a file that was never committed) and makes sure nobody, including the agent, forgets to run the tests. The file is identical for any Julia project. The workflow runs only when `.github/` sits at the root of a repository, which is why `DrivenSpin/` carries its own copy.

6.6 Your tools, same principles

Principle	**In the lab**
Version control	Analysis scripts, instrument settings, HDL sources of FPGA designs.
Tests	MATLAB's `matlab.unittest` framework [44]; simulation testbenches for FPGA designs.
Raw data is read-only	The agent never writes where measurements land.
Traceability	Save the commit hash and the settings with every measurement run.

Questions you may get

Q: Do I have to use DrWatson?
No. The principles are a fixed layout, results tied to a commit, and a pinned environment. DrWatson gives them to Julia projects in a few lines.

Q: Why not commit the results too?
They can be regenerated from code, data and parameters, which are committed; large binary files bloat the history. Archive final results that a paper depends on in a data repository with a DOI.

Q: How do I test code with random numbers?
Fix the seed (`Random.seed!`) in the test, and test statistical claims with a tolerance derived from the expected spread (Section 7).

Q: Does CI cost money?
GitHub Actions is free for public repositories; private repositories have a monthly quota of free minutes.

Read next: *Good enough practices in scientific computing* [32]; the DrWatson documentation [40], "Workflow tutorial"; the Scientific Python Development Guide [38] for the same ideas in another language.

7 Verification in physics

On the slides: exact checks (analytic solution, limiting case, structure, symmetry); consistency checks (convergence, independent route, bounds and units, injection–recovery); convergence order; two independent routes; injection–recovery; tests first; who checks the tests; a status for every claim.

Software engineering tests code against examples someone wrote down. Physics offers something stronger: statements that must hold whatever the implementation. The symbols of the driven spin used below are defined in Section 8: the excited-state population P_e, the Rabi frequency Ω, the detuning Δ, the generalised Rabi frequency $\Omega_\mathrm{R} = \sqrt{\Omega^2 + \Delta^2}$, the density matrix ρ, the times T_1 and T_2, the pure-dephasing time T_ϕ with rate $\Gamma_\phi = 1/T_\phi$, and the dissipator $\mathcal{D}$ of the master equation (16).

7.1 Exact statements

Check	Driven spin, and why it holds
Analytic solution	$P_e(t) = (\Omega^2/\Omega_\mathrm{R}^2)\sin^2(\Omega_\mathrm{R} t/2)$ from $\lvert g\rangle$, Eq. (14).
Limiting case	$\Omega = 0$: H is diagonal, populations do not move. $\Delta = 0$, $t = \pi/\Omega$: full inversion, $P_e = 1$.
Structure	$\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues of $\rho \geq 0$: the Lindblad equation preserves all three (Section 8.3); a numerical scheme need not.
Symmetry	P_e is even in Δ: Eq. (14) depends on Δ^2 only. It stays even with decoherence: the map $\rho \mapsto \sigma_z \rho^* \sigma_z$ turns the master equation (16) at Δ into the one at $-\Delta$ and leaves $\lvert g\rangle\langle g\rvert$ and all populations unchanged (checked numerically).

A symmetry test is cheap and catches a whole class of bugs: a term with the wrong sign breaks it even when every individual value looks plausible.

7.2 Consistency checks

Convergence. For a method of order p, the error at a fixed final time T scales as $\varepsilon(\delta t) \simeq C\delta t^{\,p}$ for small step δt, with C independent of δt. Halving the step therefore divides the error by 2^p:

$$p \simeq \log_2 \frac{\varepsilon(\delta t)}{\varepsilon(\delta t/2)}. \tag{8}$$

The test on the slide compares the free decay of P_e from $|e\rangle$ with the exact e^{-T/T_1} for the classical fourth-order Runge–Kutta method (RK4) with $T_1 = 1\,\mu$s, $T = 2\,\mu$s and $\delta t = 0.02$ and $0.01\,\mu$s, and demands $p = 4 \pm 0.2$. The slide's numbers were checked: the measured order is 4.01. A convergence test catches a wrong integrator, a missing term in the equation (the error then saturates at a constant), and a result reported before it has converged. The same idea applies to any cutoff: grid size, Fock-space truncation, number of Monte Carlo samples.

The test, as the stage-2 exercise sheet `exercises/stage2.md` gives it (with $\Omega = 0$, start in $|e\rangle$, $T_1 = 1\,\mu$s and $T = 2\,\mu$s):

```
ρT(dt) = last(lindblad(H, ρ0, [T]; T1, T2, dt))           # fixed-step RK4
err(dt) = abs(excited_population(ρT(dt)) - exp(-T / T1))  # free decay from |e⟩
@test log2(err(0.02) / err(0.01)) ≈ 4 atol = 0.2          # RK4: p = 4
```

The reference solution passes it. The step sizes matter: the error is 2×10^{-11} at $\delta t = 0.01\,\mu$s, still about three orders of magnitude above the accumulated rounding error; much smaller steps would measure rounding, not the method.

Independent route. Solve the same problem with a different method written by someone else, and compare. For the driven spin: the hand-written Lindblad solver against QuantumOptics.jl [53]. Note how the dephasing appears there: the slide passes the jump operator $\sqrt{1/(2T_\phi)}\,\sigma_z$, whose rate $1/(2T_\phi) = \Gamma_\phi/2$ matches the term $(\Gamma_\phi/2)\mathcal{D}[\sigma_z]\rho$ of Eq. (16). Agreement is strong evidence because independent implementations are unlikely to share an error; disagreement does not tell you which one is wrong. The same holds for derivations: check every analytic result numerically, and every numerical result against a limit you can do by hand.

Bounds and units. $0 \le P_e \le 1$; $T_2 \le 2T_1$ (Section 8.3); rates in rad/µs. A test with a known period catches a factor of 2π: with $\Omega = 2\pi$ rad/µs the resonant Rabi period is 1 µs.

7.3 Injection–recovery

The experimentalist's test of an analysis pipeline. Choose true parameters θ^*; simulate data with realistic noise; run the pipeline, which returns fitted values $\hat\theta$ with reported standard errors σ; and check

$$\left|\hat\theta - \theta^*\right| < 3\sigma \tag{9}$$

for each parameter. One seed tests that the pipeline works. Many seeds test that the error bars are honest: the *pulls* $(\hat\theta - \theta^*)/\sigma$ must be distributed as a Gaussian of zero mean and unit width. A mean away from zero is a bias; a width above one means the error bars are too small. This is the test that catches an error-bar computation that ignores correlations, a noise model that does not match the data, or a fit that converges to a wrong local minimum. Stage 4 of the hands-on (Section 8.5) is an injection–recovery test. The reference solution adds the many-seed version as a test: over 200 seeds, the pulls of T_2 must have $|\text{mean}| < 0.25$ and a standard deviation between 0.8 and 1.2.

7.4 Tests first

The sequence on the slide: you list the checks; the agent writes the tests; you read and commit them; the agent implements without touching `test/`; you read the diff. Committed tests are a fixed target the agent cannot move unnoticed: any change to them shows up in `git diff test/`. Writing them first also forces you to say what "correct" means before you see a result, which protects against accepting whatever comes out. This is the software engineers' test-driven development, and it matches the verifiable-reward training of Section 2: give the agent a check, and it iterates until the check passes.

Who checks the tests? An agent writing both code and tests grades its own homework. Tests derived from the implementation ("compute the value, paste it as expected value") check only that the code does what it does. Test against physics. Break the code on purpose: every important test must fail. Changing the code systematically and checking that some test fails is called *mutation testing*; the planted bug of block 8 is one hand-made mutation. And read every line of `test/`: it is short, and it is the part you cannot delegate.

7.5 A status for every claim

The slide's labels come from the instructor's own projects: *verified* (reproduced by an independent route), *fixed* (wrong as stated, corrected), *weakened* (true only under a narrower condition, now stated), *retracted* (does not survive), *open* (not decidable with the tools at hand). Keeping such a list in a file in the repository turns "the agent said so" into a record that a referee, or you in a year, can audit.

Questions you may get

Q: How do I test a fit where I do not know the true answer?
That is what injection–recovery is for: you create a case where you do know it, with the same noise and sampling as the real data.

Q: My result agrees with QuantumOptics.jl. Is it right?
It is much more likely to be right. Both could share a wrong model (for example the wrong sign convention for Δ), so the analytic and limiting-case tests remain necessary.

Q: What if the agent says a test is wrong?
It might be. The prompt on the slide says "If a test seems wrong, stop and tell me why". Then you decide, and the change to the test is its own commit with its reason in the message.

8 The driven spin: physics of the hands-on

On the slides: block 7 gives one card per stage: (1) Rabi oscillations, the starting point; (2) decoherence, T_1 and T_2; (3) Ramsey fringes and spin echo; (4) a synthetic experiment and fit; (5, stretch) the Jaynes–Cummings model. Each stage is worked "tests first" in four steps (Section 8.7).

This section derives every formula on the stage cards, so that every test can be explained. All formulas were checked numerically against a direct integration of the equations (Python/SciPy, independent of `DrivenSpin/`). Krantz et al. [45] is an excellent experimental-minded review of the same material for superconducting qubits.

8.1 The model

A two-level system with states $|e\rangle$ (excited) and $|g\rangle$ (ground), transition angular frequency ω_q, is driven by a field of angular frequency ω_d. In the laboratory frame,

$$H_{\text{lab}}(t) = \frac{\omega_q}{2}\sigma_z + \Omega\cos(\omega_d t)\,\sigma_x, \tag{10}$$

where Ω is set by the drive amplitude, and σ_x, σ_y, σ_z are the Pauli matrices in the basis $(|e\rangle, |g\rangle)$. Transform to the frame rotating with the drive, $|\psi'\rangle = U|\psi\rangle$ with $U(t) = \exp(i\omega_d t\sigma_z/2)$. The new Hamiltonian is $UH_{\text{lab}}U^\dagger + i\dot{U}U^\dagger$. The second term gives $-(\omega_d/2)\sigma_z$; in the first, $\sigma_+ = \sigma_-^\dagger$ and σ_- pick up phases $\mathrm{e}^{\pm i\omega_d t}$, and the drive splits into a static part $(\Omega/2)\sigma_x$ and terms oscillating at $2\omega_d$. Dropping the latter (the *rotating-wave approximation*, valid for $\Omega, |\Delta| \ll \omega_d$) leaves the course Hamiltonian

$$H = \frac{\Delta}{2}\sigma_z + \frac{\Omega}{2}\sigma_x, \qquad \Delta \equiv \omega_q - \omega_d. \tag{11}$$

Ω is the *Rabi frequency*, Δ the *detuning*. Units: $\hbar = 1$, times in µs, Ω and Δ in rad/µs.

Bloch vector. Write a state as $\rho = \frac{1}{2}(1 + \mathbf{S}\cdot\boldsymbol{\sigma})$ with the Bloch vector $\mathbf{S} = (\langle\sigma_x\rangle, \langle\sigma_y\rangle, \langle\sigma_z\rangle)$; $|e\rangle$ is the north pole $S_z = +1$, $|g\rangle$ the south pole. For $H = \frac{1}{2}\boldsymbol{\omega}\cdot\boldsymbol{\sigma}$ with $\boldsymbol{\omega} = (\Omega, 0, \Delta)$, the Heisenberg equations give a precession,

$$\dot{\mathbf{S}} = \boldsymbol{\omega}\times\mathbf{S}, \tag{12}$$

about the axis $\boldsymbol{\omega}$ at angular frequency $|\boldsymbol{\omega}| = \Omega_{\text{R}} \equiv \sqrt{\Omega^2 + \Delta^2}$, the *generalised Rabi frequency*. The excited-state population is $P_e = \langle e|\,\rho\,|e\rangle = (1 + S_z)/2$.

8.2 Stage 1: Rabi oscillations

With $\mathbf{n} = \boldsymbol{\omega}/\Omega_{\text{R}}$, Eq. (11) is $H = (\Omega_{\text{R}}/2)\,\mathbf{n}\cdot\boldsymbol{\sigma}$, and since $(\mathbf{n}\cdot\boldsymbol{\sigma})^2 = 1$,

$$U(t) = \mathrm{e}^{-iHt} = \cos\frac{\Omega_{\text{R}}t}{2} - i\sin\frac{\Omega_{\text{R}}t}{2}\,\mathbf{n}\cdot\boldsymbol{\sigma}. \tag{13}$$

The matrix element $\langle e|\,\mathbf{n}\cdot\boldsymbol{\sigma}\,|g\rangle = n_x - in_y = \Omega/\Omega_{\text{R}}$ gives, starting from $|g\rangle$,

$$P_e(t) = |\langle e|\,U(t)\,|g\rangle|^2 = \frac{\Omega^2}{\Omega_{\text{R}}^2}\sin^2\frac{\Omega_{\text{R}}t}{2}. \tag{14}$$

At resonance ($\Delta = 0$) the population oscillates fully, $P_e = \sin^2(\Omega t/2)$, with period $2\pi/\Omega$: 1 µs for $\Omega = 2\pi$ rad/µs. Off resonance the oscillation is faster and its amplitude $\Omega^2/\Omega_{\text{R}}^2$ smaller.

A resonant pulse of duration t rotates the Bloch vector about x by the *pulse area* $\theta = \Omega t$:

$$R(\theta) = \mathrm{e}^{-i\theta\sigma_x/2}. \tag{15}$$

A π-pulse ($t = \pi/\Omega$) maps $|g\rangle \to -i\,|e\rangle$; a $\pi/2$-pulse creates an equal superposition. The phase is visible in $\langle\sigma_y\rangle$: from $|g\rangle$ at $\Delta = 0$, $\dot{\mathbf{S}} = \boldsymbol{\omega}\times\mathbf{S}$ with $\mathbf{S}(0) = (0, 0, -1)$ gives $\mathbf{S}(t) = (0, \sin\Omega t, -\cos\Omega t)$, so $\langle\sigma_y\rangle(t) = \sin(\Omega t)$. This sign is the one that flips in the "bug your tests cannot see" (Section 9).

Stage-1 tests (in the starting point): Eq. (14) at many times; the π-pulse inverts at $\Delta = 0$; unitarity $U^\dagger U = 1$ and norm conservation.

The whole stage-1 library, the participants' starting point:

`DrivenSpin/src/DrivenSpin.jl`

```
"""
    DrivenSpin

A driven two-level system (a spin, or a qubit) in the frame rotating with the drive.
```

```
Conventions (see AGENTS.md): ħ = 1; times in μs, angular frequencies in rad/μs;
basis order (|e⟩, |g⟩), so |e⟩ = [1, 0] and |g⟩ = [0, 1]; σz|e⟩ = +|e⟩;
σ_ = |g⟩⟨e|; detuning Δ = ω_q − ω_d.
"""
module DrivenSpin

using LinearAlgebra

export σx, σy, σz, σm, ket_e, ket_g
export hamiltonian, propagator, excited_population

# Pauli matrices in the basis (|e⟩, |g⟩).
const σx = ComplexF64[0 1; 1 0]
const σy = ComplexF64[0 -im; im 0]
const σz = ComplexF64[1 0; 0 -1]
const σm = ComplexF64[0 0; 1 0]    # σ_ = |g⟩⟨e|

const ket_e = ComplexF64[1, 0]
const ket_g = ComplexF64[0, 1]

"""
    hamiltonian(Δ, Ω)

Rotating-frame Hamiltonian H = (Δ/2) σz + (Ω/2) σx of a spin driven at Rabi
frequency `Ω` and detuning `Δ` (both in rad/μs).
"""
function hamiltonian(Δ, Ω)
    return Δ/2 * σz + Ω/2 * σx
end

"""
    propagator(H, t)

Time-evolution operator U(t) = exp(−iHt) of a time-independent Hamiltonian `H`.
"""
propagator(H, t) = exp(-im * H * t)

"""
    excited_population(H, t)

Probability P_e(t) = |⟨e|U(t)|g⟩|^2 to find the spin in |e⟩ at time `t` (μs),
starting from |g⟩ at t = 0.
"""
excited_population(H::AbstractMatrix, t::Real) = abs2(ket_e' * propagator(H, t) * ket_g
    )

"""
    excited_population(ρ)

Excited-state population ⟨e|ρ|e⟩ of the density matrix `ρ`.
"""
excited_population(ρ::AbstractMatrix) = real(ρ[1, 1])

end # module
```

The matrices are written out in the basis $(|e\rangle, |g\rangle)$, so the basis order is visible in the code and fixed by the test $\sigma_z |e\rangle = +|e\rangle$. The propagator is the matrix exponential, exact for a time-independent

H; Eq. (14) is therefore tested against an exact computation, and the tolerance 10^{-10} in the test only absorbs rounding. The second method of `excited_population` takes a density matrix: stage 2 needs it, and the basis order makes it the $(1,1)$ element.

8.3 Stage 2: decoherence

A qubit coupled to an environment is described by its density matrix ρ. Under the Born–Markov approximations (weak coupling, short environment memory), its evolution has the Gorini–Kossakowski–Sudarshan–Lindblad form [47, 48, 49]. The course uses

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -i[H,\rho] + \Gamma_1 \mathcal{D}[\sigma_-]\rho + \frac{\Gamma_\phi}{2}\mathcal{D}[\sigma_z]\rho, \qquad \mathcal{D}[L]\rho = L\rho L^\dagger - \tfrac{1}{2}\{L^\dagger L, \rho\}, \tag{16}$$

with $\sigma_- = |g\rangle\langle e|$, the energy-relaxation rate $\Gamma_1 = 1/T_1$ and the pure-dephasing rate $\Gamma_\phi = 1/T_\phi$. The first dissipator moves population from $|e\rangle$ to $|g\rangle$ (spontaneous emission into the environment); the second randomises the phase between $|e\rangle$ and $|g\rangle$ without exchanging energy (for example, a fluctuating ω_q).

Why this form. Lindblad [47] and Gorini, Kossakowski and Sudarshan [48] showed in 1976 that Eq. (16), with any operators L and non-negative rates, is the most general Markovian generator that keeps ρ a valid density matrix: trace one, Hermitian and positive (in fact completely positive) at all times. Negative rates, or a dissipator with the wrong sign, break positivity, which is why the structure tests of stage 2 detect them.

Solving it. Take $\Omega = 0$ and write $\rho_{ee} = \langle e|\,\rho\,|e\rangle$, $\rho_{eg} = \langle e|\,\rho\,|g\rangle$. Working out Eq. (16) element by element ($\mathcal{D}[\sigma_z]$ leaves the diagonal alone and multiplies ρ_{eg} by -2),

$$\dot\rho_{ee} = -\Gamma_1\rho_{ee}, \qquad \dot\rho_{eg} = -\left(i\Delta + \frac{\Gamma_1}{2} + \Gamma_\phi\right)\rho_{eg}. \tag{17}$$

The population decays at Γ_1; the coherence at

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi}. \tag{18}$$

Since $\Gamma_\phi \geq 0$, $T_2 \leq 2T_1$: the coherence decays at least half as fast as the population, because each decay event also destroys the phase. A parameter set with $T_2 > 2T_1$ would need $T_\phi < 0$, which violates positivity; hence the specification that `lindblad` rejects it with an `ArgumentError`. $T_1 = T_2 = \infty$ switches dissipation off and must reproduce stage 1. The boundary $T_2 = 2T_1$ is allowed ($\Gamma_\phi = 0$), but computing $\Gamma_\phi = 1/T_2 - 1/(2T_1)$ in floating point can give a tiny negative number there; an implementation has to accept it.

The exercise sheet `exercises/stage2.md` fixes the interface: `lindblad(H, ρ0, ts; T1, T2, dt)` returns the density matrices at the times `ts`, integrating with fixed-step RK4 of step at most `dt`.

Stage-2 tests (stage card): at all times $\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues ≥ 0; $\Omega = 0$ from $|e\rangle$: $P_e(t) = \mathrm{e}^{-t/T_1}$; $\Omega = \Delta = 0$ from $|+\rangle = (|e\rangle + |g\rangle)/\sqrt{2}$: $|\rho_{eg}(t)| = \frac{1}{2}\mathrm{e}^{-t/T_2}$; $T_2 > 2T_1$ rejected; $T_1, T_2 \to \infty$ gives stage 1.

Numerics. Equation (16) is linear in ρ, $\dot\rho = \mathcal{L}\rho$. RK4 with step δt applies a fourth-order polynomial in $\delta t\,\mathcal{L}$ per step. Because $\mathcal{L}$ preserves trace and Hermiticity, so does the polynomial, up to rounding; positivity it preserves only approximately, with an error that shrinks with δt. The structure tests therefore test the implementation of $\mathcal{L}$ more than the integrator, and the convergence test (Section 7) tests the integrator. With the slide's parameters the measured order is $p = 4.01$.

8.4 Stage 3: Ramsey fringes and spin echo

Ramsey. Start in $|g\rangle$; apply $R(\pi/2)$ (Eq. (15)); let the qubit evolve freely for a time τ with the drive off, so $H = (\Delta/2)\sigma_z$ and the Bloch vector precesses about z at angular frequency Δ; apply $R(\pi/2)$ again; measure P_e. Pulses are taken as ideal (instantaneous) and $T_1 \to \infty$, so that $T_2 = T_\phi$. During the free evolution the equatorial component of the Bloch vector rotates by $\Delta\tau$ and shrinks by $\mathrm{e}^{-\tau/T_2}$; the second pulse maps it to

$$P_e(\tau) = \tfrac{1}{2}\left[1 + \mathrm{e}^{-\tau/T_2}\cos(\Delta\tau)\right]. \tag{19}$$

At $\tau = 0$ the two $\pi/2$-pulses make a π-pulse and $P_e = 1$. The fringes oscillate at the detuning: this is how experiments calibrate ω_q against the drive, and how they measure coherence times. Ramsey introduced the method of separated oscillatory fields in 1950 [50].

Inhomogeneous dephasing. In a real experiment the detuning differs from shot to shot by a static offset δ (slow drifts, a distribution of ω_q in an ensemble). Averaging $\cos[(\Delta+\delta)\tau]$ over a Gaussian distribution of δ with standard deviation s_δ gives $\cos(\Delta\tau)\,\mathrm{e}^{-s_\delta^2\tau^2/2}$: an extra, Gaussian decay. The coherence time measured by Ramsey is therefore usually called T_2^*, and $T_2^* \le T_2$.

Echo. Hahn's spin echo [51] inserts a π-pulse in the middle: $\pi/2$ – $\tau/2$ – π – $\tau/2$ – $\pi/2$. The π-pulse about x maps $\sigma_z \to -\sigma_z$, so the phase $\delta\tau/2$ acquired in the first half is undone in the second, whatever δ is. With ideal pulses and no decoherence the sequence returns $|g\rangle$ for every static δ, $P_e = 0$ (the total rotation is 2π about x). Averaging over δ therefore changes nothing: the echo removes static dephasing and leaves only the irreversible part, T_2. Fluctuations of δ during the sequence are not refocused, which is why echo decay measures T_2. With Markovian dephasing (Eq. (16), $T_1 \to \infty$) the coherence still decays during both halves, and the echo gives

$$P_e(\tau) = \tfrac{1}{2}\left[1 - \mathrm{e}^{-\tau/T_2}\right] \tag{20}$$

for every static δ; at $\tau = 0$ the sequence is a 2π rotation and $P_e = 0$.

Stage-3 tests (stage card and `exercises/stage3.md`): fringes at angular frequency Δ under the envelope $\mathrm{e}^{-\tau/T_2}$; echo independent of a static δ; averaged over random static δ, Ramsey decays and echo does not. The last test needs numbers: the reference tests draw 200 detunings with $s_\delta = 2\,\mathrm{rad/\mu s}$ and a fixed seed, and compare the averaged Ramsey signal at $\Delta = 0$ with $\frac{1}{2}[1 + \mathrm{e}^{-s_\delta^2\tau^2/2}]$ within 0.05, which is several times the sampling error of 200 draws. Two cheap extra tests: $R(\pi)\,|g\rangle = -i\,|e\rangle$, and $R(2\pi) = -1$, the sign of a spinor under a full rotation.

8.5 Stage 4: a synthetic experiment and its fit

Data. For each delay τ_k compute $P_e(\tau_k)$ from stage 3 and simulate N single-shot measurements: the number of shots that find $|e\rangle$ is binomial, $n_k \sim \mathrm{Binomial}(N, P_e(\tau_k))$, and the measured population is $\hat{P}_k = n_k/N$ with variance $P_e(1-P_e)/N$.

Fit. The model on the stage card is

$$P(\tau) = B + A\,\mathrm{e}^{-\tau/T_2}\cos(\Delta\tau + \varphi), \tag{21}$$

with amplitude A, offset B and phase φ as additional parameters; for ideal pulses $A = B = \frac{1}{2}$ and $\varphi = 0$, and leaving them free absorbs pulse errors and readout offsets, as in an experiment. LsqFit [55] minimises the weighted sum of squared residuals:

```
fit = curve_fit(model, τ, Phat, w, p0)   # w = 1 ./ variances (inverse variance)
coef(fit), stderror(fit)                # best-fit parameters and standard errors
```

The weights are the inverse variances, and they should come from the model, not from the data. The variance estimated from the data, $\hat{P}_k(1-\hat{P}_k)/N$, vanishes at $\hat{P}_k = 0$ or 1, and in general

gives too much weight to points that happen to fluctuate towards 0 or 1, which biases the fit. A robust recipe: fit once with data-based weights, with $\hat{P}_k$ clamped away from 0 and 1 (for example to $[1/(2N), 1 - 1/(2N)]$), then refit once or twice with variances computed from the fitted model. The starting values `p0` matter too: the fit has many local minima in Δ; take Δ from the peak of the periodogram (the squared Fourier transform) of the data, and T_2 of the order of the time span.

Why four times the shots halves the errors. Each $\hat{P}_k$ has standard deviation $\propto 1/\sqrt{N}$, and the standard errors σ of the fitted parameters scale linearly with the data errors. So $\sigma \propto 1/\sqrt{N}$, and $N \to 4N$ gives $\sigma \to \sigma/2$.

Stage-4 tests (stage card): injection–recovery with a fixed seed, fitted T_2 and Δ within 3σ of the true values; four times more shots give error bars about half as large; the script saves the fit with `produce_or_load`, so the file carries the commit.

The reference tests use $\Delta = 2\pi\,\mathrm{rad/\mu s}$, $T_2 = 1.5\,\mu\mathrm{s}$, 81 delays between 0 and $4\,\mu\mathrm{s}$ and $N = 200$ shots; with model-based weights, the pull test of Section 7 passes over 200 seeds, and the errors for $N = 800$ are half those for $N = 200$ within 15 %. The exercise also asks for a script `scripts/ramsey_fit.jl` that saves the fit with `produce_or_load`, so that the result file records the commit.

8.6 Stage 5 (stretch): the Jaynes–Cummings model

A qubit coupled to one mode of a cavity with angular frequency ω_c, photon annihilation operator a and coupling g [52]:

$$H_{\mathrm{JC}} = \omega_c a^\dagger a + \frac{\omega_q}{2}\sigma_z + g\left(a^\dagger\sigma_- + a\sigma_+\right), \tag{22}$$

with $\sigma_+ = |e\rangle\langle g|$ as before. The coupling term is already in rotating-wave form (valid for $g \ll \omega_c, \omega_q$): it only exchanges one photon for one qubit excitation. Hence the *excitation number* $N_{\mathrm{exc}} = a^\dagger a + \sigma_+\sigma_-$ commutes with H_{JC} and is conserved.

For each $n \geq 0$, H_{JC} couples only $|n, e\rangle$ and $|n+1, g\rangle$ (photon number, qubit state), with matrix element $\langle n+1, g|\, H_{\mathrm{JC}}\, |n, e\rangle = g\sqrt{n+1}$. On resonance ($\omega_c = \omega_q$), starting from $|n, e\rangle$,

$$P_e(t) = \cos^2\!\left(g\sqrt{n+1}\,t\right), \tag{23}$$

which oscillates at angular frequency $2g\sqrt{n+1}$; for $n = 0$ this is the vacuum Rabi frequency $2g$, also the splitting of the two dressed states in the spectrum.

Stage-5 tests: N_{exc} is conserved (commutator, or its expectation value along the evolution); the oscillation frequency is $2g\sqrt{n+1}$; the result converges in the photon cutoff. A subtlety for the last test: for a Fock initial state, conservation of N_{exc} makes the truncated dynamics *exact* once the cutoff includes $n+1$ photons, so convergence is trivial. A meaningful test starts from a coherent state of the field, whose photon-number distribution has a tail; then the populations (including the collapse and revival of the Rabi oscillations) converge only as the cutoff grows. In the reference tests, with $\alpha = 2$ (mean photon number $|\alpha|^2 = 4$) and times up to $20\,\mu\mathrm{s}$ at $g = 0.3\,\mathrm{rad/\mu s}$, cutoffs of 25 and 40 photons agree to 10^{-8}, while a cutoff of 5 differs by more than 10^{-2}. The coherent state is truncated without renormalising, so that the lost weight shows up as an error.

8.7 The hands-on in practice

Pairs, one person new to git typing the git commands, swapping the keyboard after each stage. Getting started: copy `DrivenSpin/` from the course repository into a new folder, `git init`, commit, and run the tests, which pass at stage 1.

The commands, from `DrivenSpin/exercises/README.md`; the installation guide has already cloned the course repository as `~/ai-course/course`:

```
cd ~/ai-course
cp -r course/DrivenSpin DrivenSpin
cd DrivenSpin
git init
git add .
git commit -m "Start from stage 1"
julia --project -e 'using Pkg; Pkg.instantiate(); Pkg.test()'
```

Start the agent *inside* `DrivenSpin/`, so that it reads `AGENTS.md`. The exercise sheets in `exercises/`, one per stage, contain the specification of each stage: the function to write, the tests, and for stage 4 the script.

Each stage in four steps:

1. Prompt: read `AGENTS.md` and `src/`; write *only* the tests for the stage in `test/runtests.jl`, from the checks on the stage card.
2. You read `git diff test/` line by line and commit the tests.
3. Prompt: implement the stage in `src/` until all tests pass; do not modify `test/`; if a test seems wrong, stop and explain why.
4. You read `git diff`, run the tests yourself, and commit.

Done early: push to a private GitHub repository with `gh repo create DrivenSpin -private -source=. -push` and add the CI file; check stage 2 against QuantumOptics.jl; stage 5; a convergence test of your time stepper. When stuck: ask for an explanation before a fix; after circles, `git restore .` and a fresh session.

9 Failure modes and responsibility

On the slides: the planted bug; "plausible is not correct"; the bug your tests cannot see; two rogues' galleries; you are the author; recording which tool was used.

9.1 The planted bug

Participants apply a patch to their own repository with `git apply`, make the Rabi plot, and run the tests. The patch changes one character: $\Omega/2 \to \Omega$ in `hamiltonian`. Its effect at $\Delta = 0$: the Hamiltonian becomes $\Omega\sigma_x$, so

$$P_e(t) = \sin^2(\Omega t) \quad \text{instead of} \quad \sin^2(\Omega t/2). \tag{24}$$

The plot still shows smooth, bounded, periodic oscillations between 0 and 1, at twice the frequency. Without a reference, the eye accepts both curves (slide "Plausible is not correct"). The π-pulse test fails at once: at $t = \pi/\Omega$ the buggy population is $\sin^2\pi = 0$ instead of 1. Off resonance the bug changes amplitude and frequency together, $P_e = \Omega^2/(\Omega^2 + \Delta^2/4)\,\sin^2(\sqrt{\Omega^2 + \Delta^2/4}\,t)$, so the analytic test fails at every Δ. Find the bug with `git diff`; undo it with `git restore .`.

The patch, which touches only the line of `hamiltonian`, so that it still applies after participants have added stages 2–4:

`DrivenSpin/exercises/planted_bug.patch`

```
diff --git a/src/DrivenSpin.jl b/src/DrivenSpin.jl
index 750071f..4e97989 100644
--- a/src/DrivenSpin.jl
+++ b/src/DrivenSpin.jl
```

```
@@ -30,7 +30,7 @@ Rotating-frame Hamiltonian H = (Δ/2) σz + (Ω/2) σx of a spin driven
    at Rabi
 frequency `Ω` and detuning `Δ` (both in rad/μs).
 """
 function hamiltonian(Δ, Ω)
-    return Δ/2 * σz + Ω/2 * σx
+    return Δ/2 * σz + Ω * σx
 end

 """
```

Participants apply it from inside their own `DrivenSpin/`,

```
git apply ../course/DrivenSpin/exercises/planted_bug.patch
```

or with `git apply -3` if their agent edited the surrounding lines. Before making the plot, its exercise sheet (`planted_bug.md`) has them delete the saved results with `rm -r data/sims/rabi`: `produce_or_load` recognises results by their parameters, not by the code that made them (Section 6), so without that step the script would load the old, correct curves. This is itself a lesson about cached results.

9.2 The bug your tests cannot see

Flip the sign of the Hamiltonian term in Eq. (16): $-i[H,\rho] \to +i[H,\rho]$. This is evolution under $-H$, equivalently backwards in time for the coherent part. All populations in the course are unchanged, because

- the Rabi formula (14) is even in Ω and in Δ;
- the Ramsey and echo results, Eqs. (19) and (20), depend on Δ only through $\cos(\Delta\tau)$ or not at all;
- the dissipators do not involve H.

So every test of stages 1–5 passes, including the comparison of populations with QuantumOptics.jl. Only phases change sign: at $\Delta = 0$, $\langle\sigma_y\rangle(t) = \sin(\Omega t)$ (Section 8.2) becomes $-\sin(\Omega t)$. The lesson on the slide: your tests decide which bugs you can see, so test every quantity you will publish. A single test of $\langle\sigma_y\rangle$, or of the sign of a Ramsey phase φ with a known detuning, closes this gap. For the demo, the reference solutions (kept with the instructor until after the course) include a patch `exercises/sign_bug.patch` that flips the sign in `lindblad`, and a final testset "Lindblad: phase of the coherence" that checks $\langle\sigma_y\rangle(t) = \sin(\Omega t)$ at $\Delta = 0$ and catches it.

9.3 Rogues' gallery

Failure	Why it happens, and the defence
Invented or misquoted reference	Plausible text with no lookup (Section 2). Open the link; check that the paper exists *and* says that.
Factor of 2, wrong sign	Conventions differ between sources (Ω vs. $\Omega/2$, $\Delta = \omega_q - \omega_d$ or the reverse). Analytic and limiting-case tests.
Test weakened to pass	Rewarded for passing checks (Section 2). Commit tests first; read every test diff.
Convention silently changed	Fourier normalisation, $\hbar = 1$, basis order, sign of Δ. Conventions in `AGENTS.md`, enforced by tests.
Invented or outdated function names	Knowledge cutoff, little Julia in the training data. Run it; pin versions in `Manifest.toml`.
Hz where rad/µs was meant	A factor of 2π that looks harmless. Units in names and docstrings; a test with a known period.
Confident summary of a paper not read	The model writes what such a paper would plausibly say. Ask for page and equation numbers; check them.
"All tests pass" that were never run	A summary is a claim like any other. Run them yourself; CI runs them anyway. Review the diff, not the summary.

9.4 You are the author

Accountability. An AI tool is not an author, and every claim in the paper is yours. Nature's 2023 editorial [56] set two ground rules that most publishers now follow: no AI tool is credited as an author, and its use is documented in the methods or acknowledgements. The American Physical Society updated its policy in June 2026 [57, 58]. It allows substantive uses (literature synthesis, data analysis, scientific reasoning, figures) with disclosure, and states that an AI program cannot be held accountable and therefore cannot be an author. The disclosure names the tool and version, how it assisted, how it was directed, and how its output was verified; AI used in the research goes into the methods section, AI-made figures into their captions, other substantive uses into the acknowledgements. Light editing needs no disclosure.

Data. When an agent reads a file, its content is sent to the provider. Unpublished data, collaborators' code and personal data leave your machine. Check what your institution and your collaborations allow, and what the provider's terms say about retention and training. Referees take note: APS forbids uploading manuscripts under review to tools whose terms allow training on them [58].

Disclosure. Read your journal's policy before you submit. The APS requirement "how its output was verified" is easy to meet if the verification is in the repository: the tests, the independent routes, the claim statuses.

Reproducibility. Record which tool and model helped, and when. A commit trailer costs nothing and is searchable:

```
Add Ramsey sequence with fringe-frequency test

Assisted-by: <agent> (<model>)
```

Some agents add a trailer themselves (for example `Co-Authored-By`); keep it. Git treats any `Key: value` lines at the end of the message as trailers; `git log --grep "Assisted-by"` finds them later.

9.5 Conclusion of the course

The six take-aways on the closing slide, with the section that explains each:

1. An LLM is a very large fit: plausible is not the same as correct (Section 2).
2. An agent closes the loop, so it can check its work, if you give it checks (Section 3).
3. Context is finite: one task per session, write the rest down (Section 4).
4. Git is the safety net: commit often, read every diff (Section 5).
5. Structure, pinned environments and CI: every result traceable and checked (Section 6).
6. Verification is the job: tests that encode physics, written first, read by you (Sections 7 and 8).

Questions you may get

Q: Do I have to disclose that I used an agent to write analysis code?
For APS journals, yes: AI used in conducting the research, such as data analysis, is disclosed in the methods section [58]. Other publishers differ; check.

Q: Can I paste my unpublished data into the agent?
Only if your institution, your collaboration and the provider's terms allow it. When in doubt, use synthetic data with the same structure (stage 4 is an example).

Q: The agent's summary says all tests pass. Is that enough?
No. Run them yourself, or let CI run them. Review the diff, not the summary.

10 Trendy things people talk about

Stub: to be extended

This section is a skeleton. Each entry has a one-paragraph starting point with a verified source, and a marker where more text should go. Marcelo's note said "like Jev", read here as the Jevons paradox; **confirm**, and add or remove entries.

10.1 The Jevons paradox

In *The Coal Question* (1865), W. S. Jevons argued that more efficient steam engines would *increase* Britain's coal consumption, not reduce it: cheaper use of a resource opens new uses. His Chapter VII, "Of the Economy of Fuel", puts it in one sentence: "It is the very economy of its use which leads to its extensive consumption" [59]. The argument carries over to code. If AI makes code cheap, we will not write the same code for less effort; we will write much more code, run more analyses and try more ideas. The demand for verification grows with it, which is the thesis of the course (Section 1) seen from economics.

[TO EXTEND] Whether the paradox holds depends on how elastic the demand is: ... Examples from the group: ... Counter-arguments: ...

10.2 Vibe coding

A term coined by A. Karpathy in a post on X in February 2025 [60]: accepting AI-written code without reading it, steering only by whether the result seems to work. It is fine for throw-away prototypes and the opposite of what this course recommends for research code, where "seems to work" is exactly the failure of Section 9.

[TO EXTEND]

10.3 Model Context Protocol (MCP)

An open protocol, introduced by Anthropic in November 2024 [30], through which an agent connects to external tools and data sources (databases, issue trackers, instrument control software) in a uniform way. In the language of Section 3: a standard for adding tools to a harness.

[TO EXTEND] Relevance for labs: an MCP server in front of an instrument or a data archive ...

10.4 Agent skills and instruction files

Covered in Section 4: `AGENTS.md` [28] and skills [29]. The trend is towards writing procedures down once, in plain files, for any agent to load.

[TO EXTEND]

10.5 Long-horizon agents

METR's measurements [23] suggest that the length of tasks agents complete reliably has doubled about every seven months since 2019. Extrapolations of this trend are a staple of AI discussions; the course's position is that longer unattended runs make automatic verification more important.

[TO EXTEND]

10.6 Reward hacking

When a model is optimised against a check, it may learn to satisfy the check without doing the task: modifying tests or scoring code [19]. This is Goodhart's law, which Tao discusses for mathematics [2]: a measure that becomes a target ceases to be a good measure.

[TO EXTEND]

10.7 Further candidates

[TO DECIDE] AI "scientists" and automated discovery claims; open-weight versus closed models; energy use of training and inference; benchmarks and their saturation; local models for confidential data.

Part II
Reference cards

11 Git cheat sheet

Start	
`git init`	Start recording in the current folder.
`git clone <address>`	Copy a repository (e.g. from GitHub), with its history.
Look (safe: changes nothing)	
`git status`	What is new, changed, staged? Run it often.
`git diff`	Changes not yet staged, line by line.
`git diff -staged`	What the next commit will contain.
`git diff -stat`	Which files changed, and how much.
`git log -oneline`	The history, one line per commit, newest first.
`git log -stat` / `git log -p <file>`	Files per commit / the story of one file.
`git show <hash>`	One commit in full: message and diff.
`git diff <hash1> <hash2>`	What changed between two snapshots.
Record	
`git add <file>`	Stage one file.
`git add -p`	Stage piece by piece; answer `y` or `n` per change.
`git add -u`	Stage changes to tracked files; new files stay out.
`git add .`	Stage everything new, changed or deleted here and below.
`git commit -m "why"`	Take the snapshot. The message says *why*.
Undo	
`git restore <file>`	Discard uncommitted edits to a file (*gone for good*).
`git restore .`	Discard *all* uncommitted edits here and below. Check first.
`git restore -staged <file>`	Unstage; the edit stays in the file.
`git revert <hash>`	New commit that undoes an old one; history kept.
Branch	
`git switch -c <name>`	New branch here, and switch to it.
`git switch <name>`	Switch to an existing branch.
`git merge <name>`	Bring `<name>` into the current branch.
`git branch -D <name>`	Delete a branch you no longer want.
Share	
`git push` / `git pull`	Send your commits / get and merge others'.

Reading a diff. `--a/` old file, `+++b/` new file; `@@ -12,6 +12,6 @@`: excerpt starting at line 12, 6 lines long, old and new; `-` removed, `+` added, space unchanged. A modified line is one `-` and one `+`.

Never without asking: `git reset -hard`, `git push -force`, `git clean -f`. They destroy work.

With an agent. (1) Commit before it starts. (2) One task, one commit. (3) Read `git diff test/` first, then the rest. (4) Let it type the commands; you read the result. Useful prompts: "Commit my manual changes before starting." "Work in modular commits: one commit per change, with the tests passing at each."

What goes in: source, scripts, tests, `Project.toml`, `Manifest.toml`, `README.md`, `AGENTS.md`. **What stays out** (`.gitignore`): raw data, generated results and plots, passwords and keys, ever.

Words. *Commit*: snapshot of the whole project, named by a hash (first 7 characters suffice). *Staging area*: the next snapshot being prepared. *Branch*: a movable name for the latest commit of a line of work. `HEAD`: what you have checked out. *Remote*: another copy, usually `origin` on GitHub.

12 Before trusting AI output: a checklist

Before the agent starts

☐ The project is under git and the last commit is clean (`git status`).
☐ The task is one task, in a fresh session.
☐ The prompt names the files, the conventions (or `AGENTS.md`), the tests that define "done", and what must not change.
☐ For anything non-trivial: the agent proposes a plan and you approve it.

Tests

☐ The tests were written (or listed) *before* the implementation, read by you line by line, and committed.
☐ They check physics, not the implementation: analytic results, limiting cases, symmetries, conservation laws, bounds, units.
☐ At least one test checks every quantity you will publish, including phases and signs, not only populations.
☐ You broke the code on purpose, and the important tests failed.
☐ Approximate comparisons near zero have an absolute tolerance (`atol`).
☐ Random numbers have a fixed seed.

Reading the change

☐ You read `git diff test/` first: no loosened tolerance, deleted assertion or special case.
☐ You read the rest of the diff, not the agent's summary of it.
☐ No unexpected change to `Manifest.toml`, conventions or units.
☐ You ran the tests yourself, or CI ran them on the pushed commit.

Numbers and fits

☐ Converged: halving the step (or raising the cutoff) does not change the result beyond the tolerance; the observed order matches the method.
☐ Cross-checked by an independent route (another package, another method, a limit by hand).
☐ Injection–recovery: the pipeline recovers known parameters within error bars; over many seeds the pulls look like a unit Gaussian.
☐ Results saved with the commit hash (e.g. `produce_or_load`), from a clean repository (no `-dirty`).

Text, derivations, references

☐ Every reference opened: it exists, and it says what is claimed.
☐ Every derivation checked numerically or symbolically by a script in the repository.
☐ Every claim has a status: verified, fixed, weakened, retracted or open.

Responsibility

☐ No data left your machine that your institution or collaboration does not allow.
☐ The tool and model are recorded (commit trailer, README).
☐ The journal's AI policy is followed: disclosure of tool, role, direction and verification.
☐ You can defend every line as your own.

Part III

References

13 Annotated references

Every entry below was opened and checked on 20 October 2026: title, authors, year and link, and every number these notes quote from it. The *Checked* line says against what (the publisher's page, Crossref, the arXiv record, or the document itself). Entries that could only be checked in part say so. Entries marked ⋆ are the best next reading for participants.

AI in science

[1] A. Ramesh Chandra, A. Mahajan, A. Sanyal, M. Wang et al. (39 authors), *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026). https://arxiv.org/abs/2610.16733

Use: Motivation slide. 50 unpublished problems; best agent 98 % (49/50), under 3 USD per problem; little change without web search; no agent-led breakthroughs.

Checked: the PDF (v1, 9 Oct 2026): abstract and Sec. on results.

[2] ⋆ T. Tao, *Mathematics in the age of AI*, arXiv:2608.16753 (2026). https://arxiv.org/abs/2608.16753

Use: Motivation slide. Essay based on his ICM 2026 public lecture. Section 7, "Proof scarcity and proof abundance", contains the quoted sentence; also discusses Goodhart's law.

Checked: the PDF (v1, 17 Aug 2026); quotation verbatim from Sec. 7.

[3] J. Becker, N. Rush, E. Barnes, D. Rein, *Measuring the impact of early-2025 AI on experienced open-source developer productivity*, arXiv:2507.09089 (2025). https://arxiv.org/abs/2507.09089

Use: Scepticism: 16 developers, 246 tasks; forecast −24 % time, perceived −20 %, measured +19 %.

Checked: arXiv record and abstract.

How LLMs work

[4] ⋆ A. Karpathy, *[1hr Talk] Intro to Large Language Models*, YouTube (22 Nov 2023). https://www.youtube.com/watch?v=zjkBMFhNj_g

Use: The best one-hour overview; pitched at a general technical audience.

Checked: video page: title, channel, upload date.

[5] ⋆ A. Karpathy, *Deep Dive into LLMs like ChatGPT*, YouTube (5 Feb 2025). https://www.youtube.com/watch?v=7xTGNNLPyMI

Use: Three and a half hours: tokenization, pre-training, post-training, RL, in the order of block 1.

Checked: video page: title, channel, upload date.

[6] ⋆ 3Blue1Brown (G. Sanderson), *Neural networks* series, in particular *Transformers, the tech behind LLMs* (Chapter 5, 2024) and *Attention in transformers, step-by-step* (Chapter 6, 2024). https://www.3blue1brown.com/topics/neural-networks

Use: Visual explanation of attention (backup slide).

Checked: series page; video pages for chapters 5 and 6 (`wjZofJX0v4M`, `eMlx5fFNoYc`), uploaded April 2024.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, *Attention is all you need*, Advances in Neural Information Processing Systems **30** (2017); arXiv:1706.03762. https://arxiv.org/abs/1706.03762

Use: The transformer; attention formula of the backup slide.

Checked: arXiv record (title, authors, 2017). The NeurIPS volume is the standard citation; the arXiv record itself names no venue.

[8] R. Sennrich, B. Haddow, A. Birch, *Neural machine translation of rare words with subword units*, arXiv:1508.07909 (2015). https://arxiv.org/abs/1508.07909

Use: Byte-pair encoding for tokenization.

Checked: arXiv record.

[9] A. Grattafiori, A. Dubey et al. (Llama Team, Meta), *The Llama 3 herd of models*, arXiv:2407.21783 (2024). https://arxiv.org/abs/2407.21783

Use: Concrete numbers for an open model: 405B parameters, 15.6T pre-training tokens, 128K-token vocabulary and context, 3.94 characters per token on English.

Checked: arXiv record (current author order starts with Grattafiori); numbers from the abstract and the HTML full text.

[10] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown et al., *Scaling laws for neural language models*, arXiv:2001.08361 (2020). https://arxiv.org/abs/2001.08361

Use: Loss as a power law in parameters, data and compute.

Checked: arXiv record.

[11] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya et al., *Training compute-optimal large language models*, arXiv:2203.15556 (2022). https://arxiv.org/abs/2203.15556

Use: Model size and data should grow together ("Chinchilla").

Checked: arXiv record.

[12] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, D. Amodei, *Deep reinforcement learning from human preferences*, arXiv:1706.03741 (2017). https://arxiv.org/abs/1706.03741

Use: Origin of learning a reward model from human comparisons.

Checked: arXiv record.

[13] L. Ouyang, J. Wu, X. Jiang, D. Almeida et al. (20 authors), *Training language models to follow instructions with human feedback*, arXiv:2203.02155 (2022). https://arxiv.org/abs/2203.02155

Use: RLHF for language models (InstructGPT); 1.3B model preferred over 175B GPT-3.

Checked: arXiv record and abstract.

[14] Y. Bai, S. Kadavath, S. Kundu, A. Askell et al., *Constitutional AI: Harmlessness from AI feedback*, arXiv:2212.08073 (2022). https://arxiv.org/abs/2212.08073

Use: Feedback from a model guided by written principles instead of human raters.

Checked: arXiv record.

[15] J. Wei, X. Wang, D. Schuurmans, M. Bosma et al., *Chain-of-thought prompting elicits reasoning in large language models*, arXiv:2201.11903 (2022). https://arxiv.org/abs/2201.11903

Use: Intermediate reasoning steps by prompting.

Checked: arXiv record.

[16] N. Lambert, J. Morrison, V. Pyatkin, S. Huang et al., *Tülu 3: Pushing frontiers in open language model post-training*, arXiv:2411.15124 (2024). https://arxiv.org/abs/2411.15124

Use: Introduces the term "reinforcement learning with verifiable rewards" (RLVR).

Checked: arXiv record and abstract ("a novel method we call…").

[17] DeepSeek-AI (D. Guo, D. Yang, H. Zhang et al.), *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning*, arXiv:2501.12948 (2025); published in Nature **645**, 633 (2025). https://arxiv.org/abs/2501.12948

Use: Reasoning (self-reflection, verification) emerging from RL on verifiable rewards.

Checked: arXiv record, including its journal reference.

[18] M. Sharma, M. Tong, T. Korbak, D. Duvenaud et al., *Towards understanding sycophancy in language models*, arXiv:2310.13548 (2023). https://arxiv.org/abs/2310.13548

Use: Sycophancy in five assistants across four tasks; humans and preference models sometimes prefer sycophantic answers.

Checked: arXiv record and abstract.

[19] S. Von Arx, L. Chan, B. Barnes, *Recent frontier models are reward hacking*, METR blog (5 June 2025). https://metr.org/blog/2025-06-05-recent-reward-hacking

Use: Models modifying tests or scoring code to raise their score.

Checked: the page: title, date, authors, quoted phrase.

Agents and context

[20] ⋆ E. Schluntz, B. Zhang, *Building effective agents*, Anthropic engineering blog (19 Dec 2024). https://www.anthropic.com/engineering/building-effective-agents

Use: Workflows versus agents; start simple.

Checked: the page: title and date. **Partial:** the page names the authors as "Erik S. and Barry Zhang"; the surname Schluntz is from secondary sources only.

[21] S. Yao, J. Zhao, D. Yu, N. Du et al., *ReAct: Synergizing reasoning and acting in language models*, arXiv:2210.03629 (2022). https://arxiv.org/abs/2210.03629

Use: The reason–act–observe loop behind agents.

Checked: arXiv record.

[22] C. E. Jimenez, J. Yang, A. Wettig, S. Yao et al., *SWE-bench: Can language models resolve real-world GitHub issues?*, arXiv:2310.06770 (2023). https://arxiv.org/abs/2310.06770

Use: 2294 real issues from 12 Python repositories, judged by their own tests.

Checked: arXiv record and abstract.

[23] T. Kwa, B. West, J. Becker, A. Deng et al., *Measuring AI ability to complete long software tasks*, arXiv:2503.14499 (2025), NeurIPS 2025. https://arxiv.org/abs/2503.14499

Use: The 50 % time horizon; doubling about every seven months since 2019.

Checked: arXiv record and abstract.

[24] ⋆ K. Hong, A. Troynikov, J. Huber, *Context rot: How increasing input tokens impacts LLM performance*, Chroma technical report (14 July 2025). https://www.trychroma.com/research/context-rot

Use: 18 models; degradation with input length; distractors; focused (∼300 tokens) versus full (∼113K) prompts; shuffled haystacks easier.

Checked: the report page (old URL `research.trychroma.com` redirects here).

[25] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape et al., *Lost in the middle: How language models use long contexts*, arXiv:2307.03172 (2023). https://arxiv.org/abs/2307.03172

Use: Information in the middle of a long context is used worst.

Checked: arXiv record.

[26] ⋆ P. Rajasekaran, E. Dixon, C. Ryan, J. Hadfield (Anthropic Applied AI team), *Effective context engineering for AI agents*, Anthropic engineering blog (29 Sep 2025). https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Use: Context as a finite resource; compaction; note-taking; sub-agents; just-in-time retrieval.

Checked: the page: title, date, authors.

[27] Anthropic, *Best practices for Claude Code*, Claude Code documentation (living document, no date or author given). https://code.claude.com/docs/en/best-practices

Use: Tool-specific, but its core is general: give the agent a way to verify its work; explore, plan, implement, commit; keep instruction files short; clear context between tasks.

Checked: the page. The former URL `anthropic.com/engineering/claude-code-best-practices` (2025 blog post) now redirects here.

[28] *AGENTS.md*, an open format for guiding coding agents, stewarded by the Agentic AI Foundation (Linux Foundation). https://agents.md/

Use: Which agents read `AGENTS.md`.

Checked: the page.

[29] B. Zhang, K. Lazuka, M. Murag, *Equipping agents for the real world with Agent Skills*, Anthropic engineering blog (16 Oct 2025). https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

Use: Skills and progressive disclosure.

Checked: the page: title, date, authors.

[30] Anthropic, *Introducing the Model Context Protocol* (25 Nov 2024). https://www.anthropic.com/news/model-context-protocol; specification at https://modelcontextprotocol.io/

Use: Trends section.

Checked: both pages.

[31] S. Willison, *Simon Willison's Weblog*. https://simonwillison.net/

Use: Frequent, careful, hands-on notes on LLM tools; good for staying current.

Checked: the site.

Research software engineering

[32] ⋆ G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, T. K. Teal, *Good enough practices in scientific computing*, PLOS Comput. Biol. **13**, e1005510 (2017). https://doi.org/10.1371/journal.pcbi.1005510

Use: The minimum set of practices for research code; short and practical.

Checked: Crossref.

[33] G. Wilson, D. A. Aruliah, C. T. Brown, N. P. Chue Hong et al., *Best practices for scientific computing*, PLOS Biol. **12**, e1001745 (2014). https://doi.org/10.1371/journal.pbio.1001745

Use: The more ambitious predecessor.

Checked: Crossref.

[34] The Turing Way Community, *The Turing Way: a handbook for reproducible, ethical and collaborative data science*. https://book.the-turing-way.org/

Use: Reference handbook on reproducibility, version control, testing.

Checked: the site; the old address `the-turing-way.netlify.app` still works.

[35] ⋆ S. Chacon, B. Straub, *Pro Git*, 2nd ed. (Apress, 2014), free online. https://git-scm.com/book/en/v2

Use: Chapters 2 (basics), 3 (branching), 10 (internals).

Checked: the site.

[36] ⋆ Software Carpentry, *Version Control with Git*. https://swcarpentry.github.io/git-novice/

Use: Self-paced git lesson for researchers.

Checked: the site.

[37] The Carpentries Incubator, *Python Testing and Continuous Integration* (alpha). https://carpentries-incubator.github.io/python-testing/

Use: Testing and CI in Python; the ideas transfer to Julia.

Checked: the site. Note: an incubator lesson, not an official Software Carpentry one.

[38] Scientific Python, *Scientific Python Library Development Guide*. https://learn.scientific-python.org/development/

Use: Packaging, testing and CI for Python research code.

Checked: the site.

[39] G. Datseris, J. Isensee, S. Pech, T. Gál, *DrWatson: the perfect sidekick for your scientific inquiries*, J. Open Source Softw. **5**, 2673 (2020). https://doi.org/10.21105/joss.02673

Use: Citation for DrWatson.

Checked: Crossref and the JOSS page.

[40] ⋆ DrWatson.jl documentation. https://juliadynamics.github.io/DrWatson.jl/stable/

Use: `produce_or_load`, `tagsave`, `savename`; the "Workflow tutorial".

Checked: the pages on saving and naming; `savename` and `dict_list` output also checked by running DrWatson.

[41] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, *Julia: A fresh approach to numerical computing*, SIAM Rev. **59**, 65 (2017). https://doi.org/10.1137/141000671

Use: Citation for Julia.

Checked: Crossref.

[42] Julia documentation, *Unit Testing* (standard library `Test`). https://docs.julialang.org/en/v1/stdlib/Test/

Use: `@test`, `@testset`; `isapprox` tolerances (behaviour checked by running Julia 1.13).

Checked: the page.

[43] julia-actions, GitHub Actions for Julia. https://github.com/julia-actions

Use: `setup-julia`, `cache`, `julia-buildpkg`, `julia-runtest`.

Checked: the organisation page and the `julia-runtest` repository.

[44] MathWorks, *matlab.unittest.TestCase* (MATLAB documentation). https://www.mathworks.com/help/matlab/ref/matlab.unittest.testcase-class.html

Use: Unit tests in MATLAB.

Checked: the page.

Physics of the toy problem

[45] ⋆ P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, W. D. Oliver, *A quantum engineer's guide to superconducting qubits*, Appl. Phys. Rev. **6**, 021318 (2019); arXiv:1904.06560. https://doi.org/10.1063/1.5089550

Use: Rotating frame, Rabi, Ramsey, echo, T_1, T_2, from an experimental viewpoint.

Checked: Crossref and arXiv record.

[46] I. I. Rabi, *Space quantization in a gyrating magnetic field*, Phys. Rev. **51**, 652 (1937). https://doi.org/10.1103/PhysRev.51.652

Use: Historical origin of the Rabi formula.

Checked: Crossref.

[47] G. Lindblad, *On the generators of quantum dynamical semigroups*, Commun. Math. Phys. **48**, 119 (1976). https://doi.org/10.1007/BF01608499

Checked: Crossref.

[48] V. Gorini, A. Kossakowski, E. C. G. Sudarshan, *Completely positive dynamical semigroups of N-level systems*, J. Math. Phys. **17**, 821 (1976). https://doi.org/10.1063/1.522979

Use: With [47]: the general form of a Markovian master equation.

Checked: Crossref.

[49] H.-P. Breuer, F. Petruccione, *The Theory of Open Quantum Systems* (Oxford University Press, 2002). https://doi.org/10.1093/acprof:oso/9780199213900.001.0001

Use: Textbook derivation of the master equation.

Checked: Crossref (the DOI is the 2007 online edition).

[50] N. F. Ramsey, *A molecular beam resonance method with separated oscillating fields*, Phys. Rev. **78**, 695 (1950). https://doi.org/10.1103/PhysRev.78.695

Checked: Crossref.

[51] E. L. Hahn, *Spin echoes*, Phys. Rev. **80**, 580 (1950).
https://doi.org/10.1103/PhysRev.80.580

Checked: Crossref.

[52] E. T. Jaynes, F. W. Cummings, *Comparison of quantum and semiclassical radiation theories with application to the beam maser*, Proc. IEEE **51**, 89 (1963).
https://doi.org/10.1109/PROC.1963.1664

Checked: Crossref.

[53] S. Krämer, D. Plankensteiner, L. Ostermann, H. Ritsch, *QuantumOptics.jl: A Julia framework for simulating open quantum systems*, Comput. Phys. Commun. **227**, 109 (2018).
https://doi.org/10.1016/j.cpc.2018.02.004

Use: The independent route of block 6.

Checked: Crossref.

[54] QuantumOptics.jl documentation. https://docs.qojulia.org/

Use: `timeevolution.master(tspan, rho0, H, J)`; `SpinBasis(1//2)` with `spinup` $= |e\rangle$.

Checked: the page on master equations; basis convention and solver checked by running QuantumOptics.jl 1.2.

[55] LsqFit.jl documentation. https://julianlsolvers.github.io/LsqFit.jl/latest/

Use: `curve_fit` with inverse-variance weights; `coef`, `stderror`.

Checked: the tutorial page; function names checked by running LsqFit 0.16.

Responsibility and policy

[56] Editorial, *Tools such as ChatGPT threaten transparent science; here are our ground rules for their use*, Nature **613**, 612 (2023). https://doi.org/10.1038/d41586-023-00191-1

Use: No AI author; document use in methods or acknowledgements.

Checked: Crossref. **Partial:** the publisher page required a login; the two rules are from consistent secondary reproductions.

[57] American Physical Society, *American Physical Society releases updated AI policy for APS journals* (17 June 2026).
https://www.aps.org/about/news/2026/06/releases-updated-ai-policy-journals

Checked: the page.

[58] ⋆ APS Journals, *Appropriate Use of AI Tools*.
https://journals.aps.org/authors/appropriate-use-ai-tools

Use: Current rules for authors and referees of APS journals.

Checked: the page (undated).

Trends

[59] W. S. Jevons, *The Coal Question* (Macmillan, London, 1865; 2nd ed. 1866). Full text of the 2nd edition: https://www.econlib.org/library/YPDBooks/Jevons/jvnCQ.html

Use: The Jevons paradox; Chapter VII, “Of the Economy of Fuel”.

Checked: the Econlib text (chapter title and quoted sentence).

[60] A. Karpathy, post on X introducing "vibe coding" (2 Feb 2025).
https://x.com/karpathy/status/1886192184808149383

Checked: **Partial:** the post did not render without login; date and wording from several consistent secondary sources.

Claims on the slides without a checkable source

- "About 0.75 words per token" (slide, block 1) is a widely quoted rule of thumb from OpenAI's help pages; the page refused automated access. Section 2 supports the number with the Llama 3 figure of 3.94 characters per token [9] instead.